

Orbit Wars — RL Formulation and Training Math

Native C++/LibTorch stack

June 8, 2026

This document specifies the policy, the reward, and the training updates used by the native trainer. The trainer supports *two* algorithms — *Group Relative Policy Optimization* (GRPO, a group baseline, Section 7) and **PPO + GAE** (a value-head critic, Appendix A) — selected by `TrainConfig::algo`. The current `docs/set-ups/1.md` run (the **v5 target actor** with the redesigned event reward) trains with **PPO + GAE**; GRPO is the alternative. Symbols map to `native/rl/config.hpp` and `native/rl/model/`.

Leading implementation note. The actively-developed implementation is the self-contained pure-PyTorch notebook `notebooks/setup1_target_ppo.ipynb` (it mirrors the native trunk/reward bit-for-bit). It has since taken the v5 actor **five** steps further, all reflected below: (i) the per-planet action is now a single **per-source Dirichlet allocation** simplex (Section 4) — each owned source emits one E -way row of ship fractions over all destinations *including itself*, the self-diagonal being **HOLD**, so over-launch is structurally impossible; (ii) destinations the sun would absorb are *hard-masked* via a reachability mask (Section 4); (iii) single-snapshot self-play is replaced by an **Elo/PFSP league** (Section 9), and the hand-coded curriculum stages are *removed* — the league *is* the curriculum (Section 6); (iv) the launch *budget* is enforced by the row simplex rather than a per-slot running deduction — one source can still fan its garrison across several destinations in a turn, but simultaneously, with no slot order to get wrong (Section 1); and (v) headings aim at the destination’s orbital *intercept* (**LEAD_TARGET**, Section 1) rather than its current position. The native C++ stack still carries the continuous- ϕ single-launch target actor; porting these is pending. Appendix B maps the **AlphaZero/MuZero family** onto this discrete actor — the planned next step beyond PPO.

Contents

1	Problem setup	2
2	Observation encoding	5
3	Policy network π_θ	7
3.1	Recommended hyperparameters: trunk vs. transformer trunk	11
3.2	Inference latency vs. configuration	12
4	Action distribution	12
5	Reward and return	14
6	Curriculum (leading: none — the Elo league <i>is</i> the curriculum)	16
7	GRPO update	16
7.1	Grouped sampling	17
7.2	Group-relative advantage (no critic)	17
7.3	Clipped policy surrogate	17
7.4	KL anchor to a reference policy	17

7.5	Total objective	18
7.6	Optimization	18
8	KL divergence and the PPO trust region	18
9	Self-play league (Elo / PFSP)	20
10	Self-play league, curriculum, and reward shaping	21
10.1	Scripted anchor bots (the difficulty ladder)	21
10.2	Curriculum: Elo-gated anchor regimes	22
10.3	Elo grounding, recalibration, and matchmaking	22
10.4	Reward shaping: production-weighted, asymmetric capture	23
11	Batched GPU simulator	24
11.1	Simulator fidelity (sim-to-real gap)	25
12	Symbols $\leftrightarrow$ config.hpp	27
A	Appendix: PPO + GAE (the current default algorithm)	28
B	Appendix: search-based policy improvement — the AlphaZero/MuZero family	28
B.1	What maps directly: AlphaZero, not MuZero	28
B.2	Obstacle 1: a combinatorial (factored) action space	29
B.3	Obstacle 2: very few simulations — Gumbel policy improvement	29
B.4	Obstacle 3: simultaneous moves and a hidden opponent	30
B.5	Targets, loss, and the training-loop change	30
B.6	Minimal first cut for Orbit Wars	31

1 Problem setup

Orbit Wars is an N -player symmetric Markov game with $N \in \{2, 4\}$. The engine sets $N = |\text{state}|$ (`num.agents`) and assigns N home planets from a 4-fold-symmetric quadrant group ($N = 2$: two opposite quadrants; $N = 4$: all four). Each turn all players *simultaneously* issue a set of fleet *launches*; the deterministic engine then moves fleets, resolves collisions/combat, rotates planets, produces ships, and (on scheduled steps) spawns comets. Scoring is a **free-for-all**: at termination player i 's score is its total ship count ships_i (planets + fleets), and the seat(s) with the maximum score receive reward +1, all others -1 (so $N = 4$ is winner-take-all among four individuals — not teams).

Training scope of this document. We currently train the **two-player special case** ($N = 2$: ego $p = 0$ against one opponent in the opposite quadrant). The native world generator places exactly two homes; the shaped reward (Section 5) uses the zero-sum margin $\text{ships}_0 - \text{ships}_1$; and each group plays a single opponent (scripted, or a frozen snapshot, Section 9). The policy (Section 3) and the GRPO update (Section 7) are N -agnostic; lifting to 4-player FFA requires only (i) 4-home world generation, (ii) replacing the zero-sum margin with the FFA return (max-score seat +1 else -1 , or a rank-based shaping), and (iii) seating ≥ 1 extra opponents per game. This 2p $\rightarrow$ 4p gap is an open modeling item, revisited in Section 5.

Target action space (current — per-source Dirichlet *allocation* actor). A turn’s action is *factored over the E planet slots* of the observation, but the per-planet action is no longer a bundle of K discrete launch slots: it is a single **allocation simplex**. A single head emits, per **owned** source planet s , a row $A[s, \cdot] \in \mathbb{R}^E$ of *fractions over all E destinations including s itself*, summing to 1:

- the **self-diagonal** $A[s, s]$ is the **HOLD** fraction — the share of s ’s ships *kept* (no launch). Targeting the source itself *is* the no-op, so there is no separate launch gate, hold column, or no-op bucket;
- each **off-diagonal** $A[s, d]$ ($d \neq s$) is the fraction of s ’s ships sent to destination d — a real planet slot, masked to the *alive and sun-reachable* destinations (the reachability mask of Section 4).

Because each source row sums to 1, the per-planet **ship budget is structural**: with garrison $S = S_s$,

$$\sum_{d \neq s} \lfloor A[s, d] S \rfloor \leq (1 - A[s, s]) S \leq S, \quad (1)$$

so **over-launch is impossible** — there is no running budget and no slot order to get wrong. This replaces the previous $K=10$ multi-launch design (a fixed bundle of $(d_{e,k}, \varphi_{e,k})$ slots decoded under a per-slot running budget), which the allocation simplex subsumes: one source can still fan its garrison across several destinations in one turn (several off-diagonal entries), but the fractions are now *simultaneous* rather than sequential, and the budget is enforced by construction rather than by decode order. The per-planet action is thus a single (B, E, E) allocation matrix A (one E -simplex row per source planet), with `n_action_params` = E per source and **no** separate ϕ head and **no** K launch slots.

A **heuristic** turns each chosen destination d into its heading: with `LEAD_TARGET=true` this aims at the destination’s orbital *intercept* (the deterministic lead math below); the static-target fallback is the straight line $\theta_{s,d} = \text{atan2}(p_y[d] - p_y[s], p_x[d] - p_x[s])$ at the destination planet’s current position. Either way the model **never learns aiming** — this removes the continuous-aiming learning problem that motivated the earlier (d^x, d^y) direction-vector design (and its passive-basin failure).

Policy parameterization (PPO: a Dirichlet over each source row). Under `POLICY_MODE="ppo"` the `dest_head` logits parameterize a **Dirichlet** on each owned source row. With per-destination logits $\ell_s \in \mathbb{R}^E$ the concentration is

$$\alpha_s = \text{softplus}(\ell_s) \cdot \text{ALLOC_KAPPA} + \epsilon, \quad A[s, \cdot] \sim \text{Dirichlet}(\alpha_s), \quad (2)$$

and the action’s log-probability and entropy **sum over the owned source planets** (unowned planets contribute nothing). `ALLOC_KAPPA` sets the overall concentration (how peaked vs. spread the allocation starts), ϵ a small floor keeping every $\alpha > 0$. A second mode `POLICY_MODE="mcts"` reuses the *same* `dest_head` logits as search *priors* and discretizes the allocation into a tree search, with the self-diagonal $A[s, s]$ serving as the **no-op branch** (this is a stub for now; the search appendix, Appendix B, is the design it targets).

Decode: allocation $\rightarrow$ engine launches. For each ego-owned source s ($u_s = 1$, $S_s > 0$) and each *off-diagonal* destination $d \neq s$ that is alive, the launch size is $n_{s,d} = \lfloor A[s, d] S_s \rfloor$ ships, sent only when $n_{s,d} \geq \text{MIN_LAUNCH_SHIPS}$. A below-threshold off-diagonal entry is simply **held** (folded

back into the garrison), *not* penalized — there is no miss/illegal-launch stick for it. Each executed launch is aimed at the orbiting intercept (Section 1). Because every destination is a real, sun-reachable planet and the row simplex bounds total dispatch by S_s , the action is **structurally legal** — over-dispatch and spraying are gone, so the old invalid-dispatch-penalty machinery and the “learn legality from a penalty / optional action mask” discussion become moot for this actor.

The destination axis is sized to $E = \text{planet_cap}$ (the observation slot count), so the target actor **forces** `max_entities = planet_cap`: in the GPU env this is $E = 48$ (the build sets `max_entities ← planet_cap` whenever `target_actor` is on). The self-targeting HOLD trick is what removes the need for any separate launch gate or hold column: keeping ships is just allocating mass to the diagonal.

Lead-target aiming (LEAD_TARGET): deterministic orbital intercept. With `LEAD_TARGET=true` the heading is no longer a straight line at the destination’s *current* position — it aims at where the orbiting destination *will be* when the fleet arrives. The destination planet d orbits the board centre C on a circle of radius r_d at a *fixed* angular velocity w , so at flight-time t (in ticks) it is at

$$P(t) = C + r_d (\cos(\phi_0 + wt), \sin(\phi_0 + wt)), \quad (3)$$

where ϕ_0 is its current orbital phase. A fleet launched from the *source* planet leaves that planet’s **surface**: a point at distance $r_{\text{src}} + 0.1$ from the source centre c_{src} along the chosen heading. It then flies straight at the speed $v = v(n)$ of Eq. (1) set by its ship count n . The deterministic *intercept time* t^* is the earliest root of

$$g(t) = \|P(t) - c_{\text{src}}\| - (r_{\text{src}} + 0.1) - vt = 0, \quad (4)$$

i.e. the straight-line distance the fleet must cover (target range minus the surface-launch offset) equals the distance it travels in t ticks. The heading is then aimed at the predicted point, $\theta = \text{atan2}(P(t^*) - c_{\text{src}})$. The static-target case ($w = 0$, or `LEAD_TARGET=false`) recovers the straight-line heading at the planet’s current position.

Intercept bug found and fixed. The previous solver missed $\sim 20\%$ of *reachable* orbiting targets (static targets were already 100%). Two faults compounded: (i) it solved Eq. (4) from the planet **centre**, dropping the surface-launch offset $r_{\text{src}} + 0.1$, so the predicted arrival was systematically early; and (ii) it used a 4-iteration *fixed-point* $t \leftarrow \|P(t) - c_{\text{src}}\|/v$, which is **not a contraction** when $r_d|w| > v$ (a fast-orbiting target outruns the iterate), so it failed to converge on exactly the reachable-but-hard cases. The fix accounts for the surface offset and replaces the fixed-point with a robust root of g in Eq. (4) — more iterations / a bracketed bisection on $g(t)$ (which is sign-changing and well behaved on the reachable range). This recovers $\sim 94\text{--}96\%$ of reachable orbiting targets. The residual $\sim 4\text{--}6\%$ is **sun occlusion** — the straight path to the earliest intercept clips the central sun disk, so no straight shot at *that* intercept exists — a *planning* limit (the reachability mask of Section 4 would reject such a shot), not an aiming bug.

Superseded continuous actor (target_actor=false). The target actor *supersedes* the earlier **continuous** actor, which is retained behind the `--target-actor` flag (`target_actor=false`, the config default). There each slot e emitted a fixed block of $3K$ *continuous* controls ($K = 5$ fleet slots, three scalars each), so the action was a 40×15 matrix $a \in [0, 1]^{E \times 3K}$ ($E = 40$). Fleet slot k of planet e was a triple $(d_{e,k}^x, d_{e,k}^y, \phi_{e,k}) \in [0, 1]^3$: a *direction vector* (d^x, d^y) , recentred to $[-1, 1]$ as $2d - 1$ and mapped to a launch angle $\theta_{e,k} = \text{atan2}(2d_{e,k}^y - 1, 2d_{e,k}^x - 1)$ ($0\text{--}360^\circ$, continuous —

no bins), and a *ship fraction* $\phi_{e,k}$ mapped to 0–100% of the launching planet’s current ships. A planet could thus dispatch up to $K = 5$ fleets in one step, each to an independently chosen heading and size; the engine processed the K launches in slot order, deducting ships *sequentially*. That actor was **not** masked: it emitted the full 40×15 block every step and was meant to learn the dynamic legal subset from an invalid-dispatch penalty (the desired output for a non-existent slot being the zero action; dispatching from an unowned/empty planet or *over-dispatching* a planet’s K fractions past 100% being penalized “everywhere”). An optional hard mask (**action_mask_policy**) could instead force the per-planet legal mask $m_e u_e \mathbf{1}[S_e > 0]$ onto the action and drop masked slots from the log-prob/entropy sums; both the penalty-learned legality and that mask applied *only* to this continuous actor. The continuous actor’s direction-vector parameterization made the heading *linear in relative position* — attention over the planet tokens (Section 3) could produce a target-minus-self vector directly, whereas a linear head cannot turn a relative position into $\text{atan2}(\cdot)$ a scalar heading would require — but learning that aiming proved hard, which is precisely what the target actor sidesteps by emitting a destination slot and aiming via the heuristic above.

2 Observation encoding

For ego player p and state s , the encoder (**native/core/encode.hpp**) emits four arrays: the per-planet entity matrix $X \in \mathbb{R}^{E \times F}$ ($E = 40$ planet slots, $F = 101$ features each); the board-summary globals $g \in \mathbb{R}^G$ ($G = 10$, a *separate* input — **not** replicated into each row); the entity mask $m \in \{0, 1\}^E$ ($m_e = 1$ iff slot e holds a real entity); and the action mask $u \in \{0, 1\}^E$ ($u_e = 1$ iff e is an ego-owned planet with > 0 ships). *Neither m nor u gates the actor* (Section 3): m only hides padding slots inside the attention block, and u is *both* an input feature (body row 10, below) and the key used to *decode* launches and score the invalid-dispatch penalty (Section 5); the dynamic legal action subset is otherwise *learned* from that penalty, not hard-masked. Each planet row $X_e \in \mathbb{R}^{101}$ is 11 *body* features (unchanged) followed by a 90-feature *incoming-threat block*; the body block is enumerated below and the threat block in Section 2.

Per-planet body features (rows 0–10). Each value is normalized into roughly $[-1, 1]$. Here S is the planet’s ship count, ϱ its radius, d_c its distance to board centre, ω the board angular velocity, and $\ell(n) = \ln(1 + n) / \ln 1000$ the shared ship-count log-compressor.

#	Feature	Encoded value	Meaning
0	position x	$p_x/100$	horizontal position on the 100×100 board, $[0, 1]$
1	position y	$p_y/100$	vertical position, $[0, 1]$
2	orbital speed	$ \omega d_c/6$ (0 if static)	tangential orbit speed, relative to $v_{\max}=6$
3	spin sign	+1 ccw / −1 cw / 0	rotation direction; 0 if stationary or a comet
4	radius	$\varrho/3$	capture / collision disk size
5	production	$\text{prod}/5$	ships made per tick (the reward-defining quantity)
6	ships	$\ell(S)$	garrison size, log-compressed
7	ownership code	0 neu / 1 ego / 2– N enemy	owner, ego-relative (stable under seat rotation)
8	dist. to centre	$d_c/70.71$	radial distance from the sun ($70.71 = \frac{\sqrt{2}}{2} \cdot 100$, half-diagonal)
9	comet flag	$\mathbb{K}[\text{comet}]$	1 for a comet, else 0
10	action mask u_e	$\mathbb{K}[\text{ego} \wedge S > 0]$	1 iff ego-owned with ships (also the decode/penalty key)

Why feed both radius and production? For a normal planet they are redundant ($\varrho = 1 + \ln(\text{prod})$), but we feed both so the net need neither invert the $\ln$ nor re-derive the reward-defining quantity. *Why an explicit comet flag?* A *stationary* body has speed 0 and spin sign 0, but an *orbiting* planet

and a *comet* both move, so velocity alone cannot separate them; the flag also marks where the radius law breaks (a comet carries the fixed COMET_PRODUCTION= 1 at an unrelated radius). The comet’s remaining lifetime is **deliberately hidden by the engine** (agents may not reconstruct the spawn/expiry schedule), so this binary flag is the only observable comet signal.

Board-summary globals $g \in \mathbb{R}^{10}$. One small vector summarising the whole board, embedded once and broadcast to the action heads (Section 3) rather than copied into every planet row. With Σ a board-wide sum, n_p the planet count, and ℓ the same log-compressor:

#	Encoded value	Meaning
0	step/T	game phase (fraction of the T -step episode elapsed)
1	10ω	board angular velocity (orbit rate, scaled)
2	$\ell(\Sigma \text{ ego ships})$	ego total garrison ships (log)
3	$\ell(\Sigma \text{ enemy ships})$	enemy total garrison ships (log)
4	n_p^{ego}/n_p	ego share of all planets
5	n_p^{enemy}/n_p	enemy share of all planets
6	n_p^{neutral}/n_p	neutral share of all planets
7	$\ell(\Sigma \text{ ego fleet})$	ego in-flight (fleet) ships (log)
8	$\ell(\Sigma \text{ enemy fleet})$	enemy in-flight (fleet) ships (log)
9	$\min(n_p, E)/E$	planet-slot occupancy (how full the E rows are)

This game is a **free-for-all**: there are no allies; an inbound fleet is either the ego’s own (a reinforcement) or an enemy’s, distinguished by the owner sign in the threat block (next).

Incoming-fleet threat block (per planet, baked into the row). A purely *reactive* policy cannot defend if it cannot see what is coming. For each planet we therefore bake a compact list of *its own* inbound fleets directly into that planet’s feature row. First we project every in-flight fleet to the planet it will hit, and when. A fleet at position o with unit heading $\hat{d} = (\cos \alpha, \sin \alpha)$ and ship count ν moves at speed

$$v(\nu) = \min\left(v_{\max}, 1 + (v_{\max} - 1)\left(\frac{\ln \max(1, \nu)}{\ln 1000}\right)^{1.5}\right), \quad v_{\max} = 6, \quad (5)$$

matching the engine’s fleet dynamics. For a planet at c with radius ϱ , the ray $o + t\hat{d}$ enters the planet disk at

$$t_{\text{ca}} = -\langle o - c, \hat{d} \rangle, \quad p^2 = \|o - c\|^2 - t_{\text{ca}}^2, \quad t_{\text{hit}} = t_{\text{ca}} - \sqrt{\varrho^2 - p^2} \quad (\text{if } t_{\text{ca}} \geq 0, p^2 \leq \varrho^2). \quad (6)$$

The fleet is assigned to the planet with the smallest $t_{\text{hit}} \geq 0$ (dropped if the ray first crosses the central sun — a *fixed* disk of radius SUN_RADIUS = 10, constant all game — or hits nothing), with ETA $\tau = t_{\text{hit}}/v(\nu)$ in ticks.

Two ranked views per planet (tuneable counts). Among the fleets assigned to a planet we keep two short lists: the N_SOON= 15 with the smallest ETA (most *imminent*) and the N_BIG= 15 with the largest ship count (most *dangerous*); the lists may overlap. Each of the N_SOON+N_BIG = 30 slots contributes 3 features

$$[\text{sgn}, \tau, \tilde{\nu}], \quad \tilde{\nu} = \frac{\ln(1 + \nu)}{\ln 1000}, \quad (7)$$

i.e. *who it belongs to* (sgn = +1 ego reinforcement / -1 enemy), *arrival time* as the ETA τ in ticks (scaled by 1/THREAT_ETA_SCALE, THREAT_ETA_SCALE = 100), and *ship count* $\tilde{\nu}$. Empty slots are all

zero — $\text{sgn} = 0$ there already marks “no fleet,” so $\tau = 0$ is unambiguous — giving the row’s $30 \times 3 = 90$ threat features, which occupy entity-row indices 11–100. The slots are laid out as two contiguous ranked blocks; slot k (1-indexed) occupies the three indices $11 + 3(k - 1)$, $12 + 3(k - 1)$, $13 + 3(k - 1)$:

Slot k	Ranked by	sgn idx	ETA idx	ships idx
1	soonest ETA (1st)	11	12	13
2	soonest ETA (2nd)	14	15	16
$\vdots$	$\vdots$ (N_SOON= 15)	$\vdots$	$\vdots$	$\vdots$
15	soonest ETA (15th)	53	54	55
16	largest fleet (1st)	56	57	58
$\vdots$	$\vdots$ (N_BIG= 15)	$\vdots$	$\vdots$	$\vdots$
30	largest fleet (15th)	98	99	100

(slots 1–15 are the N_SOON= 15 soonest-arriving, slots 16–30 the N_BIG= 15 largest; per slot the three values are $\text{sgn} \in \{+1 \text{ mine}, -1 \text{ enemy}, 0 \text{ empty}\}$, $\text{ETA} = \tau/100$ ticks, $\text{ships} = \tilde{\nu} = \ell(\nu)$). Because the list is attached to the destination planet’s *own* row, that planet directly sees “what is coming at *me* — how soon, how big, and from whom” — local defensive awareness baked into the *observation*, with no separate fleet tokens. (The *encoder* has no attention; the *policy network* does add one cross-planet attention block, Section 3, so a planet can read the others’ positions to aim at them.) The projection ignores planet rotation but is computed *identically* in C++ and Python (parity-tested bit-for-bit), so the natively trained policy behaves identically when served from pure Python.

3 Policy network π_θ

The network (`native/rl/model/policy_net.hpp`) maps the per-planet entity matrix $X \in \mathbb{R}^{E \times F}$ (body + incoming-threat features, Section 2) to a per-planet block of action parameters (`n_action_params` = E per source for the target actor’s single (B, E, E) allocation matrix A ; $3K$ for the superseded continuous actor). The $G = 10$ board-summary globals enter through a *small separate path*: a one-layer embedding $g' = \text{ReLU}(W_g g + b_g) \in \mathbb{R}^{d_g}$ ($d_g = 32$), broadcast to every planet and concatenated at the action heads — so the trunk itself stays purely per-planet (per-row). The actor heads carry **no separate pointer head**; a value head (the PPO critic) is built *only* when `use_value_head` is set (the `set-ups/1.md` run is PPO, so it is built; GRPO needs no critic). Let $d = \text{hidden}$.

Trunk pipeline. The trunk maps the projected rows through, in order: input projection $\rightarrow$ **one masked cross-planet self-attention block** $\rightarrow$ an optional residual GLU block (variant A) $\rightarrow n_{\text{res}}$ pre-LayerNorm residual MLP blocks $\rightarrow$ a final LayerNorm. The projection $h_e^{(0)} = W_{\text{proj}} x_e + b_{\text{proj}}$ and the MLP/GLU blocks are per-row (weights shared across planets); the attention block is the *only* cross-row mixing.

(1) Masked self-attention (the aiming mechanism). A single-head attention over the E planet tokens, computed on the pre-LayerNormed tokens $\tilde{h} = \text{LN}(h^{(0)})$:

$$A = \text{softmax}\left(\frac{(\tilde{h}W_Q)(\tilde{h}W_K)^\top}{\sqrt{d}} + \mathcal{M}_{\text{ent}}\right) (\tilde{h}W_V), \quad h^{(1)} = h^{(0)} + A W_O, \quad (8)$$

where the key-mask $\mathcal{M}_{\text{ent}}[\cdot, j] = -\infty$ for *padding* planets ($m_j = 0$, the `entity_mask`) so live planets never attend to empty slots. This block is what lets a planet read another planet’s *position* and point a launch *at* it — a purely per-row trunk cannot represent a targeted heading (it was added

precisely to unblock aiming). It is now **toggleable** (USE_ATTENTION, an A/B like `use_glu`): when off, the whole block (its `attn_{q,k,v,o}` and `ln_attn`) is not built and the trunk is *purely per-planet*. In the *target* actor aiming is already heuristic, so the block’s remaining job is *cross-planet board reasoning* — comparing candidate destinations’ ownership/garrisons/threats to choose the destination slot — which makes the ablation meaningful (at $d=512$ it is $\sim 1.05M$ of the $\sim 5.3M$ parameters; the value head still pools over planets either way).

(2) **Optional residual GLU** (variant A only, `use_glu`; Eq. 10, Fig. 2): $h \leftarrow W_o((W_v h) \odot \sigma(W_g h))$.

(3) **Pre-LayerNorm residual MLP blocks** (n_{res} of them): $\mathcal{M}(z) = z + W_b \text{ReLU}(W_a \text{LN}(z)) + b_b$, then a **final LayerNorm**. The pre-norm is not cosmetic: without it the deep trunk drove the head pre-activations large, the sigmoid squash saturated, and the aiming gradient vanished — pre-LN keeps activations bounded so the heads stay in the squash’s responsive range. So

$$h_e = \text{LN}\left(\underbrace{(\mathcal{M} \circ \dots \circ \mathcal{M})}_{n_{\text{res}}}[\circ \mathcal{B}_{\text{GLU}}](h_e^{(1)})\right) \in \mathbb{R}^d, \quad e = 1, \dots, E, \quad (9)$$

($\mathcal{B}_{\text{GLU}}$ present only in variant A; $h^{(1)}$ is the post-attention token). The `use_glu` A/B (GLU vs. an extra residual MLP) is the trunk comparison; everything else is identical. Two masks appear and must not be confused: **entity_mask** (m_e , live-vs-padding) gates *attention* above; **action_mask** ($u_e \mathbf{1}[S_e > 0]$, owned-with-ships) gates the *action* (the optional policy mask, Section 4). The board-wide signal is the small broadcast embedding g' concatenated at the heads.

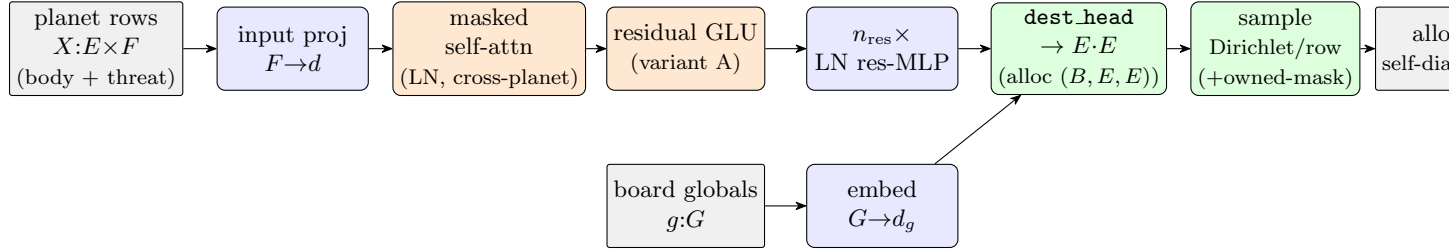


Figure 1: Policy network data flow. The **trunk is shared by both actors**; only the heads differ. Planet rows are projected, then mixed by one **masked cross-planet self-attention** block (pre-LayerNorm; **entity_mask** hides padding planets) — the only cross-row step. The result passes through the shared per-row trunk: variant A inserts one residual GLU block (Fig. 2) before the n_{res} *pre-LayerNorm* residual MLP blocks (variant B: MLP blocks only), then a final LayerNorm. The G board globals are embedded separately (g' , dim d_g) and broadcast to the heads. **Target actor (current)**: a *single* allocation head — **dest_head** maps $[h_e; g']$ to $E \cdot E$ logits (reshaped (B, E, E)), one E -way row per source planet over all destinations *including itself*. Under `POLICY_MODE="ppo"` each owned source row is a Dirichlet $A[s, \cdot] \sim \text{Dirichlet}(\text{softplus}(\ell_s) \text{ALLOC_KAPPA} + \epsilon)$, masked to alive, sun-reachable destinations, with the self-diagonal $A[s, s]$ as HOLD (Section 4); log-prob/entropy sum over owned sources. Each off-diagonal $A[s, d]$ decodes to $\lfloor A[s, d] S_s \rfloor$ ships ($\geq \text{MIN_LAUNCH_SHIPS}$, else held), aimed by a heuristic (atan2 at the orbital *intercept*, Section 1). (`POLICY_MODE="mcts"` reuses the same logits as search priors with the self-diagonal as the no-op branch; stub.) **Continuous actor (superseded)**: a two-layer MLP mean head maps $[h_e; g']$ to the $3K$ means μ_e and a switchable second head/vector to the $3K$ log-stds, defining a squashed Gaussian whose samples are the K fleet triples (d^x, d^y, ϕ) . A value head (the PPO critic) is built only when `use_value_head` is set; no pooling, no pointer head.

Residual GLU block. For an input $z \in \mathbb{R}^d$, with three $d \times d$ weight matrices (gate W_g , value W_v , output W_o),

$$\text{GLU}(z) = (W_v z + b_v) \odot \sigma(W_g z + b_g), \quad \mathcal{B}(z) = z + W_o \text{GLU}(z) + b_o \in \mathbb{R}^d, \quad (10)$$

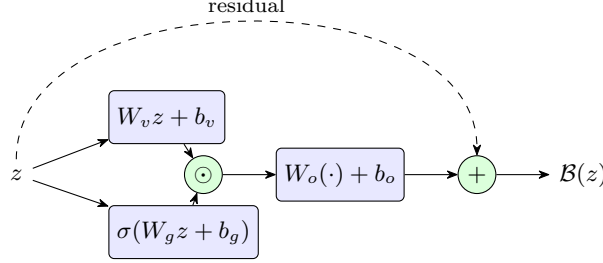


Figure 2: The residual GLU block $\mathcal{B}$ (Eq. 10): a value branch $W_v z$ and a sigmoid gate $\sigma(W_g z)$ are multiplied elementwise ($\odot$), projected by W_o , and added back to the input (the residual skip). In trunk variant A one such block (its own $d \times d$ weights) follows the input projection, before the residual MLP stack.

where σ is the logistic sigmoid and $\odot$ is elementwise. The sigmoid gate lets the block pass, attenuate, or suppress each feature channel conditionally (e.g. amplify the incoming-threat channels of Section 2 when a planet is under attack); the residual keeps optimization easy. It is applied per planet row and shares weights across rows just like the rest of the trunk. Whether this gating earns its keep over plain stacked MLPs (variant A vs. B) is exactly the comparison this experiment is set up to answer.

Target-actor head (a single (B, E, E) allocation head). The trunk above is **unchanged**; only the head differs. It reads $h'_e = [h_e; g'] \in \mathbb{R}^{d+d_g}$ (trunk token concatenated with the broadcast globals embedding) and emits one allocation row per planet:

dest.head: a single $\text{Linear}(d + d_g \rightarrow E)$ producing, for each source planet s , the destination logits $\ell_s \in \mathbb{R}^E$ (reshaped over the batch to (B, E, E) — one E -way row per source, one logit per candidate destination *including s itself*). This is the *only* actor head: there is **no** ϕ head and **no** K launch slots, since the ship fractions *are* the row of the allocation simplex and **HOLD** is the self-diagonal. *Calm init:* weights scaled by `init_mu_scale`= 0.02 and the diagonal (self/**HOLD**) bias set positive (off-diagonals near 0), so the Dirichlet starts strongly biased to “hold” with a near-uniform spread over destinations, then learns to commit.

Under `POLICY_MODE="ppo"` these logits parameterize a per-source Dirichlet ($\alpha_s = \text{softplus}(\ell_s) \text{ALLOC_KAPPA} + \epsilon$); under `POLICY_MODE="mcts"` they are search priors with the self-diagonal as the no-op branch (stub). The continuous $E \times 3K$ heads below are *not* built in target mode. Sampling (the per-source Dirichlet over the destination simplex) is in Section 4; the decode into engine launches in Section 4.

Continuous action heads (mean, and a switchable log-std) — continuous-actor variant. The heads read $h'_e = [h_e; g'] \in \mathbb{R}^{d+d_g}$ (trunk token concatenated with the broadcast globals embedding). The *mean* head is a **two-layer MLP**, $\mu_e = W_{\mu 2} \text{ReLU}(W_{\mu 1} h'_e + b_{\mu 1}) + b_{\mu 2} \in \mathbb{R}^{3K}$ (`mu.h1`→`ReLU`→`mu.h2`) — nonlinear, not a single affine map. The log-std ς_e is supplied in one of **two switchable ways** (`ModelConfig::std_state_dependent`), kept as an explicit A/B because they trade off expressiveness against training stability:

State-dependent (a head): $\varsigma_e = W_\varsigma h_e + b_\varsigma \in \mathbb{R}^{3K}$ — the spread depends on the board, so the policy explores where the board is open/uncertain and commits (small σ) where the move is forced (e.g. a clear defensive launch). More expressive, but can destabilize (it may drive $\sigma \rightarrow 0$ in “confident” states and stop exploring).

State-independent (a vector): $\varsigma_e \equiv \varsigma \in \mathbb{R}^{3K}$, one learnable vector shared across all planets and states — the standard, stable PPO default.

Either way, $\sigma_e = \exp(\text{clip}(\varsigma_e, \varsigma_{\min}, \varsigma_{\max}))$, the clip keeping σ from collapsing to 0 or exploding. **Exploration anneal (explore→exploit hand-off).** The entropy bonus pushes ς *up* to the cap, so with a fixed cap σ pins at its maximum for the whole run: the sampled action $\varphi(\mu + \sigma \varepsilon)$ stays noise-dominated, the per-trajectory advantage is then $\approx$ uncorrelated with the mean, and the policy-gradient terms *cancel* — a frozen policy that never exploits (observed empirically: $\text{adv_std} = 1$ yet $\text{kl} \rightarrow 0$). The fix is a two-phase schedule on the cap. PHASE 1 (the first `sigma_decay_iters` iterations) *forces* the cap down from $\varsigma_{\max}$ to $\varsigma_{\max}^{\text{end}}$ (e.g. $\sigma : 1.0 \rightarrow 0.05$), so the action quickly becomes mean-determined and the gradient attributable. PHASE 2 releases the cap to $\varsigma_{\max}^{\text{post}}$ (`logstd_max_post`) and hands σ to the state-dependent log-std head, free within $[e^{\varsigma_{\min}}, e^{\varsigma_{\max}^{\text{post}}}]$ (e.g. $[0.01, 0.2]$): it re-widens where the board is uncertain and tightens where the move is forced. (With `sigma_decay_iters`= 0 the cap instead decays linearly over the whole run, or not at all if $\varsigma_{\max}^{\text{end}} = \varsigma_{\max}$.) The mean head and trunk are identical across the two, only the source of ς differs, and ς is trained by the same surrogate+entropy gradients (Section 4). Each head emits $3K = 15$ numbers — the per-component parameters of the K fleet triples (d^x, d^y, ϕ) ; sampling and the $[0, 1]$ squash are in Section 4, decoding into engine launches in Section 4. The mean head is a 2-layer MLP (above) rather than a single linear map — the extra nonlinearity was added alongside attention while chasing the aiming gradient; the discrete pointer/target actor of the previous design is retired together with target mode.

Calm initialization (init_mu_scale, init_phi_bias). A deep from-scratch trunk would otherwise drive the freshly initialised mean head to large, arbitrary pre-squash values, so the policy would either spray launches everywhere or saturate the squash before learning anything. Two init knobs tame the start: the final mean-head weights are scaled by `init_mu_scale`= 0.02 so every output begins $\approx$ its bias regardless of trunk depth, and the ϕ (ship-fraction) component’s bias is set to `init_phi_bias`= -4 so $\varphi(-4) \approx 0.018$ — with the initial σ only $\sim 14\%$ of slots commit a launch at step 0. Less calm (smaller $|\text{bias}|$, larger scale) means more early commits and more mistakes to learn from; too calm and the policy is inert and learns nothing.

Layer inventory (variant A). Modules are `nn.Linear` except the `ln_*` which are `nn.LayerNorm`. Names match `policy_net.cpp`.

Module	Layers (name: shape)	Count
Input projection	<code>proj:F→d</code>	1
Self-attention	<code>attn_{q,k,v,o}:d→d</code>	4
LayerNorm (pre-attn, per-res, final)	<code>ln_attn, ln_resi, ln_out:d</code>	$n_{\text{res}}+2$
Residual GLU (variant A)	<code>glu_{gate,val,out}:d→d</code>	3
Residual MLP $\times n_{\text{res}}$	<code>resi_{a,b}:d→d</code>	$2n_{\text{res}}$
Board-globals embed	<code>g_embed:G→d_g</code>	1
<i>Target actor (current — single allocation head, no ϕ head):</i>		
Allocation head	<code>dest_head:d+d_g→E</code> (reshape (B, E, E) , Dirichlet/row)	1
<i>Continuous actor (superseded):</i>		
Mean head (MLP)	<code>mu_h1:d+d_g→d, mu_h2:d→3K</code>	2
Log-std (switchable)	<code>logstd_head:d+d_g→3K</code> or vector $\varsigma:3K$	1
Value head (PPO only): in-proj	<code>val_in:d+d_g→d</code>	0/1
residual MLP $\times \text{VALUE_RES_BLOCKS}$	<code>val_res_{a,b}:d→d, val_ln:d</code>	$3n_v$
readout	<code>val_out:d→1</code>	0/1

No pointer head. The value head (the critic) is built *only* when `use_value_head` is set — it is present for the PPO + GAE run (`set-ups/1.md`) and absent for GRPO. Exactly one actor’s heads are built, selected by `target_actor`. Variant B replaces the GLU block with an extra residual MLP block (no gating). With $F=101$ (11 body + 90 threat), $G=10$, $d_g=32$, and the `set-ups/1.md` target-actor scale $d=512$, $n_{\text{res}}=6$ this is on the order of a few M parameters. The pure-Python serving twin (`orbit_wars.rl/agents/ppo_policy.py`) must mirror this trunk, board path, and heads bit-for-bit for the Kaggle submission to reproduce the natively trained policy; it serves the actor only (no critic at inference).

3.1 Recommended hyperparameters: trunk vs. transformer trunk

The actor’s **trunk** is configurable via `ARCH` $\in \{\text{"trunk"}, \text{"transformer"}\}$: the legacy path (default `"trunk"` in `setup1_target_ppo.ipynb`) and a genuine multi-head transformer stack (default `"transformer"` in `setup1_transformer_ppo.ipynb`). **Both feed the identical action heads and value head** (Section 3); only the per-row mixing in between differs. The **TRUNK** path is the masked cross-planet self-attention block plus n_{res} residual MLP blocks described above; the **TRANSFORMER** path is a stack of pre-LayerNorm multi-head-attention + ResNet-MLP blocks, bracketed by ResNet-MLP stem/head ends. Recommended settings:

Knob	TRUNK (ARCH="trunk", legacy)	TRANSFORMER (ARCH="transformer")
hidden width	HIDDEN= 512	HIDDEN= 256 (CPU/Kaggle) / 512 (GPU train)
depth	N_RES_BLOCKS= 6	N_TX_LAYERS= 4
attention	USE_ATTENTION=True (one self-attn layer)	N_HEADS= 8 (HIDDEN div. by N_HEADS)
MLP / gating	USE_GLU=True, D_G= 32	TX_MLP_RATIO= 4
stem / head	—	N_STEM_RES= 1, N_HEAD_RES= 1
params	~6.4M	~3.8M (h= 256) / ~15M (h= 512)
forward cost	(reference: h= 512 trunk)	~340 MFLOP/fwd (h= 256) $\approx$ h= 512 trunk; h= 512 $\sim 3\times$
target	GPU training	CPU/Kaggle inference (h= 256); GPU training (h= 512)

The **TRUNK** block is the original single cross-planet self-attention layer followed by N residual MLP blocks (~6.4M params at the settings above). The **TRANSFORMER** block is a stack of L pre-LN multi-head-attention + ResNet-MLP layers with ResNet-MLP stem/head ends; at the **CPU/Kaggle**

target HIDDEN= 256, N_TX.LAYERS= 4 it is $\sim 3.8\text{M}$ params and ~ 340 MFLOP per forward — roughly the *same* forward cost as the $h=512$ trunk, yet a true 4-layer multi-head transformer. HIDDEN= 512 ($\sim 15\text{M}$ params, $\sim 3\times$ the forward cost) is fine for **GPU** training. **Sizing rationale:** the Kaggle CPU probe measured ~ 42 GFLOP/s on 2 vCPU, so at $h=256/L=4$ a single forward (~ 340 MFLOP) is on the order of *tens of ms* — within the per-turn inference budget — which is why $h=256$ is the recommended transformer width for the served Kaggle agent.

3.2 Inference latency vs. configuration

The served Kaggle agent runs **one forward per turn on a single game state** ($B=1$, $E=48$ planet slots) on a 2-vCPU CPU box (probe: 0.04 TFLOP/s fp32 matmul, 10.9 GB/s copy). Crucially this forward is *compute-bound*, not memory-bound: every per-planet linear is applied across all $E=48$ planet tokens, so each layer is a $(48\times h)(h\times h)$ matmul with arithmetic intensity $\sim 17\text{--}20$ FLOP/byte — far above the Kaggle ridge point $0.04\text{ TFLOP/s} / 10.9\text{ GB/s} \approx 3.7$ FLOP/byte. (Cross-check: the $h=512$ trunk forward, ~ 0.5 GFLOP, runs at 4.3 ms locally = 0.12 TFLOP/s $\approx 53\%$ of the local matmul peak — a utilization only a compute-bound kernel reaches.) Latency therefore scales with *matmul throughput*, so the Kaggle estimate below is the measured local $B=1$ time $\times$ the local/Kaggle matmul ratio $0.235/0.040 \approx 5.9$ (equal fraction-of-peak on the two x86 boxes).

Config	arch	h	depth	params	local ms (2 thr)	Kaggle est. ms ($B=1$, 2 vCPU)
trunk-tiny	trunk	128	4 res	0.39 M	0.7	~ 4
trunk-small	trunk	256	6 res	1.69 M	1.8	~ 10
trunk-prod	trunk	512	6 res	6.52 M	4.3	~ 25
tx-S	transformer	128	2 lyr	0.61 M	1.2	~ 7
tx-M (rec.)	transformer	256	4 lyr	3.86 M	3.5	~ 21
tx-M-deep	transformer	256	6 lyr	5.44 M	5.1	~ 30
tx-L (GPU)	transformer	512	4 lyr	15.2 M	10.5	~ 61

Even the largest 15M-parameter transformer is ~ 61 ms/turn on the Kaggle CPU, and the recommended served widths — $h=256$ trunk (~ 10 ms) or $h=256$ transformer (~ 21 ms) — leave wide headroom under any per-turn budget. **Inference latency is not the binding constraint**; model size should be chosen by training compute and policy quality, with $h=512$ reserved for GPU training (where the same forward is sub-millisecond). Numbers are CPU $B=1$; the batched GPU rollout during training amortizes the per-call overhead across thousands of envs per forward.

4 Action distribution

For the **target actor**, the per-planet action is a single **allocation simplex** per owned source planet — the row $A[s, \cdot] \in \Delta^{E-1}$ of ship fractions over all E destinations *including s itself*, with the self-diagonal $A[s, s]$ the HOLD share (planets independent of one another). The **leading actor** (POLICY_MODE="ppo") makes each owned source row a **Dirichlet** over that simplex. The policy thus factorizes as a product over the owned source planets, $\pi_\theta(a \mid s) = \prod_{s: \text{owned}} \text{Dirichlet}(A[s, \cdot] \mid \alpha_s)$, and the log-prob and entropy correspondingly **sum over the owned source planets** (below). One structural **mask** encodes a heuristic the model would otherwise have to relearn:

Reachability $R_{s,j}$ (sun occlusion). A straight shot $s \rightarrow j$ is absorbed by the central sun (disk centre $c_\odot$, fixed radius $r_\odot=10$) iff the segment crosses that disk before reaching j . With $\hat{u} =$

$(p_j - p_s) / \|p_j - p_s\|$, $t = \langle c_\odot - p_s, \hat{u} \rangle$, $p^2 = \|c_\odot - p_s\|^2 - t^2$, the shot is blocked iff $t \geq 0$, $p^2 \leq r_\odot^2$, and $0 \leq t - \sqrt{r_\odot^2 - p^2} < \|p_j - p_s\|$ (the *same* ray/disk projection as the threat encoder, Section 2). Then $R_{s,j} = \mathbb{K}[\text{not blocked}]$, with the self-diagonal $R_{s,s} = 1$ (HOLD is always reachable). R is recomputed from the encoded positions (p_x, p_y = body rows 0, 1), so it is identical in rollout and update with *no* extra trajectory buffers. There is **no** ϕ -bucket validity mask: the row simplex already bounds dispatch by the garrison, and below-threshold off-diagonals are simply held (Section 1), not illegal.

For each owned source s the masked Dirichlet concentration and its sample are

$$\alpha_{s,j} = \text{softplus}(\tilde{\ell}_{s,j}) \cdot \text{ALLOC_KAPPA} + \epsilon, \quad \tilde{\ell}_{s,j} = \ell_{s,j} \text{ if } (j=s) \vee (m_j \wedge R_{s,j}), \text{ else } -\infty, \quad (11)$$

$$A[s, \cdot] \sim \text{Dirichlet}(\alpha_s), \quad \sum_j A[s, j] = 1, \quad (12)$$

where ℓ_s is the source- s row of the `dest_head` logits, the $-\infty$ entries force $\alpha \rightarrow 0$ so unreachable/padding destinations get *zero* mass while the self-diagonal (HOLD) and reachable destinations keep theirs. The joint log-probability and entropy sum over **owned** source planets (unowned planets are forced to all-HOLD and excluded):

$$\log \pi_\theta(a \mid s) = \sum_{s: \text{owned}} \log \text{Dirichlet}(A[s, \cdot] \mid \alpha_s), \quad (13)$$

$$\mathcal{H}[\pi_\theta(\cdot \mid s)] = \sum_{s: \text{owned}} \mathcal{H}_{\text{Dir}}(\alpha_s). \quad (14)$$

At evaluation the allocation is the Dirichlet mean $A[s, j] = \alpha_{s,j} / \sum_{j'} \alpha_{s,j'}$ (no sampling). The alternative `POLICY_MODE="mcts"` reuses the *same* logits as search priors and discretizes the allocation into a tree search, with the self-diagonal as the no-op branch — the prerequisite for the search-based policy improvement of Appendix B (stub for now).

Continuous- ϕ target variant (native, TargetActorDist). The native stack still carries the older per-slot squashed-Gaussian φ actor (a destination categorical $\times$ a squashed-Gaussian $\varphi_{e,k} = \varphi(z_{e,k})$, $z_{e,k} \sim \mathcal{N}(\mu_{e,k}, \sigma_{e,k}^2)$, $\sigma_{e,k} = e^{\varsigma_{e,k}}$, $\varphi(z) = 1/(1 + e^{-z})$, summed over slots); the slot's ϕ contribution to its log-prob is $\log \mathcal{N}(z_{e,k}; \mu_{e,k}, \sigma_{e,k}^2) - \log(\varphi_{e,k}(1 - \varphi_{e,k}))$ (the last term the squash Jacobian, $z_{e,k} = \text{logit } \varphi_{e,k}$) and to its entropy $\varsigma_{e,k} + \frac{1}{2} \log 2\pi e$. The reachability mask R applies there too (it gates the destination factor). Porting it to the per-source Dirichlet allocation head is pending. The owned-planet mask is *always* applied for either target actor (replacing the $E \times 3K$ actor's "sum over all slots, invalid-penalty drives unowned to no-op" rule).

Continuous actor (superseded). The retired continuous actor was a **squashed diagonal Gaussian** over the $E \times 3K$ controls. For each component $a_{e,k,j}$ ($j \in \{d^x, d^y, \phi\}$) the network emitted a mean $\mu_{e,k,j}$ and log-std $\varsigma_{e,k,j}$, drew $z_{e,k,j} \sim \mathcal{N}(\mu, \sigma^2)$ with $\sigma = e^\varsigma$, and squashed $a_{e,k,j} = \varphi(z_{e,k,j})$. With the components conditionally independent, $\log \pi_\theta(a \mid s) = \sum_{e,k,j} [\log \mathcal{N}(z_{e,k,j}; \mu, \sigma^2) - \log(a_{e,k,j}(1 - a_{e,k,j}))]$ and $\mathcal{H} = \sum_{e,k,j} (\varsigma_{e,k,j} + \frac{1}{2} \log 2\pi e)$. There the sums ran over *all* E slots (no mask), the invalid-dispatch penalty — not a mask — driving non-existent/unowned slots toward "no launch," unless the optional `action_mask_policy` forced $m_e u_e \mathbf{1}[S_e > 0]$ and dropped masked slots from both sums. The Gaussian was over the Cartesian direction (d^x, d^y), so there was no $0/360^\circ$ wrap discontinuity.

How the destination and ϕ heads are learned (no separate target). No head has labels; all are trained *only* by the policy-gradient surrogate (PPO + GAE for the current run, Appendix A; or GRPO, Section 7) and the entropy bonus, through the dependence of $\log \pi_\theta$ on them. The **destination logits** get the standard policy-gradient $\partial \log \text{softmax}(\hat{\ell}_e)[d_e]/\partial \ell_e = \text{onehot}(d_e) - \text{softmax}(\hat{\ell}_e)$, so an advantage step pushes mass toward the destination chosen when $\hat{A} > 0$ and away when $\hat{A} < 0$. The ϕ **mean/log-std** story is the squashed-Gaussian one below. Per component, with the standardized sample $u = (z - \mu)/\sigma$,

$$\frac{\partial \log \mathcal{N}}{\partial \mu} = \frac{u}{\sigma}, \quad \frac{\partial \log \mathcal{N}}{\partial \varsigma} = u^2 - 1 \quad (\text{since } \sigma = e^\varsigma). \quad (15)$$

A policy-gradient step moves each parameter along $\hat{A} \cdot \partial \log \pi_\theta / \partial (\cdot)$ (plus the entropy term), where $\hat{A}$ is the GAE per-step advantage (PPO) or the group-relative advantage (GRPO). For the ϕ **mean**: a sampled ϕ that beat its baseline ($\hat{A} > 0$) pulls μ toward the fraction taken, one that underperformed pushes it away. For the **log-std**: the factor $u^2 - 1$ is positive when the draw landed *far* from the mean ($|z - \mu| > \sigma$) and negative when *close*, so a far-out action that *helped* ($\hat{A} > 0$) **widens** σ (explore more there), a far-out action that *hurt* **narrows** it (exploit), and a near-mean action that helped also narrows it (commit to the good mean). The entropy bonus adds a constant $+c_{\mathcal{H}}$ to the log-std gradient ($\partial \mathcal{H} / \partial \varsigma = 1$), a steady upward pressure against premature collapse of σ . Because $\varsigma_e = W_\varsigma h_e$ is produced by the log-std head, these gradients flow on into W_ς and the shared trunk — that is how the network learns *where* on the board to be exploratory versus decisive. (In the state-independent variant the same gradients instead accumulate over all slots into the one shared ς , tuning a single global exploration scale.) Adam applies the update; ς is clipped to $[\varsigma_{\min}, \varsigma_{\max}]$ for numerical safety.

Decode: action $\rightarrow$ engine launch. Target actor. For each ego-owned source planet s (ownership $u_s = 1$, ships $S_s > 0$) and each *off-diagonal* destination $d \neq s$ that is alive, the launch size is $n_{s,d} = \lfloor A[s, d] S_s \rfloor$ ships, executed when $n_{s,d} \geq \text{MIN_LAUNCH_SHIPS}$; a below-threshold off-diagonal entry (and the self-diagonal **HOLD** share $A[s, s]$) keeps its ships in the garrison and is *not* penalized. Because each source row is a simplex, the dispatched ships $\sum_{d \neq s} n_{s,d} \leq (1 - A[s, s]) S_s \leq S_s$ never exceed the garrison — the budget is structural, with no running deduction and no slot order. Each executed launch is aimed by the corrected orbital intercept of Section 1: $\theta_{s,d} = \text{atan2}(P(t^*) - c_{\text{src}})$ under **LEAD_TARGET** (else the straight line at the destination’s current position). Because every destination is a real, sun-reachable planet and the row simplex bounds total dispatch by S_s , over-dispatch and phantom/unowned launches cannot arise — there is no invalid-launch rate to penalize. **Continuous actor (superseded).** There the K fleet triples were emitted in slot order; fleet k launched $n_{e,k} = \lfloor \phi_{e,k} S_e \rfloor$ ships at angle $\theta_{e,k} = \text{atan2}(2d_{e,k}^y - 1, 2d_{e,k}^x - 1)$, the engine deducting ships *sequentially*; committed fleets that failed to execute (phantom slots, unowned planets, or past the ship budget) were exactly the x_t counted by that actor’s invalid-dispatch penalty.

5 Reward and return

The current `set-ups/1.md` reward is an **event-based** reward with *four* channels — outcome, capture, dispatch, and a production milestone — that **entirely replaces** the legacy production-shaping reward (the production-margin term P , the valid-launch carrot V , the miss-penalty stick M , the invalid-dispatch penalty I , and the enemy-suppression term S are all *off* in this run: `prod_reward_weight`, `valid_launch_reward`, `miss_launch_penalty`, `illegal_launch_penalty`, `enemy_growth_weight` are all set to 0). The four live channels (`RolloutConfig`, used identically by the CPU and GPU rollouts of Section 11) are:

1. Outcome O — a flat-then-decaying “flip of winning.” At termination the game is scored by total ships ($o_i = +1$ win, 0 draw, -1 loss). The outcome is worth ± 1000 (`win.bonus=loss.penalty=1000`), held **flat for the first 100 steps** and then *decaying symmetrically* thereafter. With episode length L and $\delta_o = \text{win_decay} = \text{loss_decay} = 0.995$ and $L_0 = \text{decay_start_step} = 100$,

$$O = \text{sign}(\text{margin}) \cdot 1000 \cdot \delta_o^{\max(0, L-L_0)}. \quad (16)$$

The symmetric decay is a “flip of winning”: it rewards a *decisive* game without making a slow loss cheaper than a fast one in the opening (both win and loss are flat through the first L_0 steps, so an early scramble is never penalized for taking time). Draw is 0 . The outcome applies in *all* stages of the 4-stage curriculum (Section 6).

2. Capture C . $+30$ for each ego planet *gained* this step and -30 for each planet *lost* (an owner flip to/from ego among alive slots), with $w_c = \text{capture_reward} = 30$:

$$C_t = w_c (\# \text{gained}_t - \# \text{lost}_t). \quad (17)$$

3. Dispatch D — a bounded “get moving” carrot. $+1$ for each of the **first 50 valid launches** of the game — a committed launch whose straight-line heading actually lands on a planet (the same closed-form ray-vs-disk projection, with sun occlusion, used for the threat features of Section 2). With $w_d = \text{dispatch_reward} = 1$ and budget $N_d = \text{dispatch_count} = 50$, and $\sum_{<t} v$ the valid launches so far,

$$D_t = w_d \cdot \min(v_t, \max(0, N_d - \sum_{<t} v)). \quad (18)$$

(It counts *valid* launches, not all launches: ships thrown into empty space earn nothing.)

4. Production milestone P — bring back the economy reward. $+20$ each time the ego’s **total ship count** (planets + in-flight fleets), s , crosses a *doubling* threshold starting at `prod_milestone_base` = 100 (i.e. 100, 200, 400, ...). Let the milestone level be

$$m(s) = \lfloor \log_2(s/\text{base}) \rfloor + 1 \quad (s \geq \text{base}), \quad m(s) = 0 \quad (s < \text{base}), \quad (19)$$

and let m_{prev} be the highest level reached so far. The step reward is $P_t = w_m \cdot \max(0, m(s_t) - m_{\text{prev}})$ with $w_m = \text{prod_milestone_reward} = 20$, then $m_{\text{prev}} \leftarrow \max(m_{\text{prev}}, m(s_t))$. The $\max(0, \cdot)$ and the monotone m_{prev} make it **monotonic**: *losing* ships never claws reward back, so the channel clearly rewards a growing economy. The old “fleet hits a planet” reward is **dropped** — it rewarded ship depletion (anti-win).

Algorithm: PPO + GAE per-step assembly. The `set-ups/1.md` run trains with **PPO + GAE** (a value-head critic and GAE, Appendix A), *not* GRPO. The per-step reward is the sum of the three dense channels, $r_t = C_t + D_t + P_t$, while the outcome O is *scattered onto the episode’s last active step* (so GAE propagates it backward as a return, rather than a flat offset). Both r_t and O are divided by $\sigma_R = \text{reward_scale} = \text{ppo_reward_scale} = 100$ before GAE, which shrinks the value targets to a fittable range for the MSE critic; advantages are re-normalized afterward, so the policy objective is unchanged. The per-game return is

$$R = O + C + D + P, \quad C = \sum_t C_t, \quad D = \sum_t D_t, \quad P = \sum_t P_t. \quad (20)$$

(The GRPO path of Section 7 would instead broadcast the group-normalized R to every step; the same four channels feed it, but PPO is the algorithm for this run.)

Legacy production-shaping reward (off in this run). For reference, the previous reward was a *smooth bounded* per-game form $R_i = O_i + P_i + V_i - I_i - M_i + S_i$ with: a capped production-margin term $P = \text{clip}(\sum_t \delta_{\Pi}^t \gamma^t w_{\Pi} \Delta \Pi_0(t), \pm P_{\max})$ on ego production $\Pi_0(s) = \sum_{p:\text{owner}=0} \text{prod}(p)$; a capped valid-launch *carrot* $V = \text{clip}(\sum_t \gamma^t w_v v_t, 0, V_{\max})$; an uncapped invalid-dispatch penalty $I = \sum_t \gamma^t \rho x_t$ on committed-but-failed launches x_t (the continuous actor’s legality signal); an uncapped *miss* stick $M = \sum_t \gamma^t \rho_m m_t$ on legal launches that land on nothing ($m_t = \text{executed} - \text{valid}$); and an enemy-suppression term $S = -\sum_{t \geq x_0} \gamma^t w_{ey} t$ that rewarded bending the opponent’s production growth down. All of these carry weight 0 in the current run and are listed only for completeness; their config knobs survive in `RolloutConfig`.

Note — 2p vs. 4p objective. The outcome O (and the implicit margin) are the two-player ($N = 2$) zero-sum forms. In $N = 4$ free-for-all the objective is *ordinal* — be the maximum of four — which is not zero-sum: $\text{sign}(\text{margin})$ would become the FFA win/lose of Section 1 (max-score seat +1 else −1, or a rank-based shaping). The algorithm machinery (advantage, surrogate, KL) is unchanged — only R in Eq. (16) is redefined.

6 Curriculum (leading: none — the Elo league *is* the curriculum)

The leading implementation has no hand-coded stages. Every iteration the opponent is PFSP-sampled from the Elo league (Section 9) over *all* pool members — the two scripted anchors (`random`, `starter`) plus the historical snapshots — so the schedule *emerges from skill*. The learner starts rated at the `random` anchor (`ELO_INIT = 0`) and PFSP feeds it the nearest-rated opponents, so it practices against `random` first, shifts to `starter` as its rating climbs past it, then to ever-stronger snapshots — reproducing the old stage progression organically, with no boundaries to tune. The only schedule knobs left are `TOTAL_ITERS`, the snapshot cadence `SELFPLAY_REFRESH`, and a short `SNAPSHOT_WARMUP` that keeps the pool anchors-only until the learner is worth snapshotting. All opponents share the one reward of Section 5 (the outcome O always applies), and the Elo update happens every iteration regardless of which opponent was drawn.

Legacy native 4-stage schedule. The native trainer still uses a **4-stage, iteration-driven** curriculum (enabled by `stage3_iters > 0`): the stage advances by *iteration count* and only the **opponent** changes. With its flags (`stage1_iters = 0`, `stage2_iters = 100`, `stage3_iters = 300`, `total_iters = 1400`): Stage 1 (noop) skipped; Stage 2 vs. `random` (iters 1–100); Stage 3 vs. `starter` (iters 101–400); Stage 4 self-play+starter+random (iters 401–1400, drawn 50/25/25 via `selfplay_prob = 0.5`). Its self-play opponent is a single *frozen snapshot* (`clone_frozen`), rebuilt on Stage-4 entry and refreshed every `ckpt_every_late = 100` iters — a lagging mirror. The notebook’s league/no-stages design supersedes this (the explicit stages become PFSP weight over the same opponents); the legacy step-fraction 3-stage schedule remains available when `stage3_iters = 0`.

7 GRPO update

GRPO is the trainer’s *alternative* algorithm (the `set-ups/1.md` run uses PPO + GAE, Appendix A). It replaces the value baseline of actor–critic methods with a *group-relative* baseline: the policy plays a group of trajectories from the *same* start against the *same* opponent, and each trajectory’s return is centered against its group.

7.1 Grouped sampling

For iteration we draw N groups (`num_groups`); group j fixes an initial state $s_0^{(j)}$ (a world) and an opponent, and rolls out G trajectories (`group_size`) under the current behavior policy $\pi_{\theta_{\text{old}}}$ (stochastic). The only within-group variation is the policy’s own action sampling, so the group mean is a valid, low-variance baseline. The env count is $N \cdot G$ (`num_envs`). For every transition we store (X, m, u, g) , the action $a_{i,t}$, the behavior log-prob $\log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})$, and the reference log-prob $\log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t})$ (Section 7.4).

7.2 Group-relative advantage (no critic)

Let $\mathcal{G}_j$ be the trajectories of group j , with mean and standard deviation

$$\mu_j = \frac{1}{G} \sum_{i \in \mathcal{G}_j} R_i, \quad \varsigma_j = \sqrt{\frac{1}{G} \sum_{i \in \mathcal{G}_j} (R_i - \mu_j)^2}. \quad (21)$$

The advantage of trajectory i (group $j = j(i)$), *broadcast to all of its timesteps*, is

$$\hat{A}_i = \begin{cases} \frac{R_i - \mu_j}{\varsigma_j + \varepsilon}, & \text{whiten_advantage} = \text{true}, \\ R_i - \mu_j, & \text{otherwise,} \end{cases} \quad (22)$$

with floor $\varepsilon = \text{adv_eps}$. There is no learned $V(s)$ and no temporal-difference or GAE term — the entire baseline is Eq. (22).

7.3 Clipped policy surrogate

With the per-step importance ratio

$$\rho_{i,t}(\theta) = \exp\left(\log \pi_{\theta}(a_{i,t} \mid s_{i,t}) - \log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})\right), \quad (23)$$

(each log-prob the entity sum of Eq. (13)), the surrogate is the PPO clip objective

$$\mathcal{L}^{\text{clip}}(\theta) = - \frac{1}{\sum_i T_i} \sum_{i,t} \min\left(\rho_{i,t} \hat{A}_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) \hat{A}_i\right), \quad (24)$$

with clip range $\epsilon = \text{clip}$.

7.4 KL anchor to a reference policy

A reference policy π_{ref} (the fixed behavior-cloning checkpoint) anchors training, which counteracts the empirically observed drift of plain PPO away from the strong BC behavior. We penalize the unbiased, non-negative k_3 estimator of $\text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}})$. With $\Delta_{i,t} = \log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t}) - \log \pi_{\theta}(a_{i,t} \mid s_{i,t})$,

$$\widehat{\text{KL}}_{i,t} = \exp(\Delta_{i,t}) - \Delta_{i,t} - 1 \quad (\geq 0), \quad \mathcal{L}^{\text{KL}}(\theta) = \frac{1}{\sum_i T_i} \sum_{i,t} \widehat{\text{KL}}_{i,t}. \quad (25)$$

7.5 Total objective

Combining the surrogate, the KL anchor (weight $\beta = \text{kl_beta}$) and an entropy bonus (weight $c_{\mathcal{H}} = \text{ent_coef}$, from Eq. (14)):

$$\mathcal{L}(\theta) = \mathcal{L}^{\text{clip}}(\theta) + \beta \mathcal{L}^{\text{KL}}(\theta) - c_{\mathcal{H}} \overline{\mathcal{H}}[\pi_{\theta}] \quad (26)$$

where $\overline{\mathcal{H}}$ is the mean entropy over transitions.

7.6 Optimization

For each iteration: collect the grouped trajectories under $\pi_{\theta_{\text{old}}} = \pi_{\theta}$; compute returns (16) and advantages (22); then for `update_epochs` passes over `minibatches` shuffled minibatches, take Adam steps on (26) with learning rate $\eta = \text{lr}$, $\epsilon_{\text{Adam}} = \text{adam_eps}$, and global gradient-norm clip `max_grad_norm`. Advantages are computed once per iteration (off $\pi_{\theta_{\text{old}}}$); the ratio (24) and KL (25) are recomputed each minibatch under the live π_{θ} .

8 KL divergence and the PPO trust region

PPO (Appendix A) is the algorithm for the `set-ups/1.md` run. Its defining idea is a *trust region*: keep each policy update close, in KL, to the behavior policy that generated the data, so the off-policy importance-sampling correction stays valid. This section collects the KL definitions, the probability ratio, the three sample estimators of KL, and how the trainer uses the KL signal alongside the clip fraction. (The KL anchor of Eq. (25) is a *separate*, explicitly-penalized KL to the frozen reference policy π_{ref} ; here we treat the *old-vs-new* KL that PPO controls implicitly through clipping and monitors as a diagnostic.)

Definition. For discrete distributions p, q over the same support, the Kullback–Leibler divergence is the expected log-ratio

$$\text{KL}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_{x \sim p} \left[\log \frac{p(x)}{q(x)} \right] \geq 0, \quad (27)$$

with equality iff $p = q$; it is *not* symmetric ($\text{KL}(p \parallel q) \neq \text{KL}(q \parallel p)$ in general). For the factored Orbit Wars policy of Section 4 the joint KL is the *sum* of the per-source Dirichlet KLs over owned planets, since KL is additive over independent factors: $\text{KL}(\pi_{\theta} \parallel \pi_{\theta_{\text{old}}}) = \sum_{s: \text{owned}} \text{KL}(\text{Dirichlet}(\alpha_s) \parallel \text{Dirichlet}(\alpha_s^{\text{old}}))$.

Probability ratio. PPO constrains the step through the per-sample importance ratio between the candidate policy π_{θ} and the behavior policy $\pi_{\theta_{\text{old}}}$ that collected the transition,

$$r_t(\theta) = \frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} = \exp(\log \pi_{\theta}(a_t \mid s_t) - \log \pi_{\theta_{\text{old}}}(a_t \mid s_t)), \quad (28)$$

the same $\rho_{i,t}$ of Eq. (24) (each log-prob the per-source Dirichlet sum over owned planets of Eq. (13)). At collection time $\theta = \theta_{\text{old}}$ so $r_t = 1$; as the update epochs drift θ away, r_t moves off 1 and the $\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta})$ grows.

Three sample estimators of KL (Schulman). The KL in Eq. (27) is an expectation over $\pi_{\theta_{\text{old}}}$, so it can be estimated from the *same* samples already drawn during the rollout. Writing $r = r_t(\theta)$ for the ratio Eq. (28) and $\log r = \log \pi_\theta - \log \pi_{\theta_{\text{old}}}$, the three standard Monte-Carlo estimators of $\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)$ are

$$k_1 = \mathbb{E}_{\pi_{\theta_{\text{old}}}}[-\log r] = \mathbb{E}_{\pi_{\theta_{\text{old}}}}\left[\log \frac{\pi_{\theta_{\text{old}}}}{\pi_\theta}\right] \quad (\text{unbiased; high variance; can be negative}), \quad (29)$$

$$k_2 = \frac{1}{2} \mathbb{E}_{\pi_{\theta_{\text{old}}}}[(\log r)^2] \quad (\text{low variance; biased; always } \geq 0), \quad (30)$$

$$k_3 = \mathbb{E}_{\pi_{\theta_{\text{old}}}}[(r - 1) - \log r] \quad (\text{unbiased; lower variance; always } \geq 0). \quad (31)$$

k_1 is the literal definition Eq. (27) estimated by sampling — exactly unbiased, but its per-sample term $-\log r$ is signed, so a finite-sample average is noisy and *can come out negative* even though the true KL cannot. k_3 is the control-variate fix: $(r - 1)$ has expectation $\mathbb{E}_{\pi_{\theta_{\text{old}}}}[r] - 1 = 0$ under $\pi_{\theta_{\text{old}}}$, so adding it cancels variance without adding bias, and the convex inequality $r - 1 \geq \log r$ makes every term ≥ 0 . (k_3 is the same $\exp(\Delta) - \Delta - 1$ form used for the reference-anchor penalty, Eq. (25), with $\Delta = \log r$ there pointed at π_{ref} .) k_2 matches k_3 to second order ($\frac{1}{2}(\log r)^2 \approx (r - 1) - \log r$ for small $\log r$) and is a cheap, always-positive proxy.

What the trainer logs. The training loop logs the k_1 estimator as its KL signal,

$$\text{approx_kl} = \text{mean}(\log \pi_{\theta_{\text{old}}} - \log \pi_\theta) = \text{mean}(\text{old_logp} - \text{new_logp}) = \mathbb{E}[-\log r], \quad (32)$$

averaged over the minibatch transitions (computed in the update inner loop alongside the surrogate). It serves two purposes: (i) **policy-step sizing / early-stopping** — a large **approx_kl** means the epoch has moved the policy too far off the data it was collected on, so the update epochs can be cut short before $\pi_{\theta_{\text{old}}}$ -vs- π_θ drift invalidates the ratio; and (ii) it is the **diagnostic paired with the clip fraction**

$$\text{clipfrac} = \mathbb{E}[\mathbb{1}(|r_t - 1| > \epsilon)], \quad \epsilon = \text{CLIP} = 0.2, \quad (33)$$

the fraction of samples whose ratio left the clip band $[1 - \epsilon, 1 + \epsilon]$. (Because k_1 is signed, a small *negative* **approx_kl** early in an epoch is normal noise, not an error.)

Clipped surrogate and the KL $\leftrightarrow$ clip relation. PPO does not penalize this old-vs-new KL directly; it bounds it by *clipping* the ratio in the surrogate

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)\right], \quad (34)$$

the maximized form of the minimized $\mathcal{L}^{\text{clip}}$ of Eq. (24). The min/clip removes the incentive to push r_t beyond $1 \pm \epsilon$: once a sample’s ratio crosses the band (it joins **clipfrac**) its surrogate gradient is gated off, so the policy stops moving on that sample. Thus **approx_kl** and **clipfrac** rise together — KL growth is what *activates* the clip, and a spiking **clipfrac** with a large **approx_kl** is the signal that the trust region is binding and the epoch should stop. This implicit-KL control (clipping) is distinct from, and runs alongside, the explicit KL *anchor* to π_{ref} of Eqs. (25)/(26): clipping keeps each step near *last iteration’s* policy, while the anchor keeps the whole run near the *frozen BC reference*.

9 Self-play league (Elo / PFSP)

The leading implementation replaces single-snapshot self-play with a **rated league** (notebook **League**). The pool holds two *anchored* scripted opponents (**random** at fixed Elo 0, **starter** at fixed Elo 600 — they pin the rating scale) plus a growing set of *frozen policy snapshots*, each with a floating rating r_m ; the learner carries its own rating r_θ .

Opponent sampling (PFSP — the whole pool, every iteration). There are no stages: at *every* iteration the single rollout opponent is drawn from *all* pool members m (anchors *and* snapshots) by *prioritized fictitious self-play*, with weight $w_m = f(p_m) + \epsilon_0$, where

$$p_m = \left(1 + 10^{(r_m - r_\theta)/400}\right)^{-1} \quad (35)$$

is the learner’s expected score against m (the logistic Elo expectation) and f is the PFSP curve — $f(p) = p(1 - p)$ (PFSP_MODE="even", default: prefer near-equal opponents) or $f(p) = (1 - p)^P$ ("hard": prefer opponents not yet beaten) — with a uniform floor ϵ_0 (PFSP_FLOOR) on every member for anti-forgetting (so the scripted anchors keep a residual probability even once the learner has outgrown them). This single weighting is what makes the Elo ladder double as the curriculum (Section 6): a low-rated fresh learner gets the nearest anchors, a high-rated one gets the strong snapshots. **One** opponent is used per rollout, so the ego win+ $\frac{1}{2}$ draw fraction over the batch is a clean match score $S \in [0, 1]$. (The earlier **selfplay_prob** snapshot-vs-anchor split is retired — the unified PFSP weight subsumes it.)

Elo update. After *every* rollout against the drawn opponent m , with K-factor K (ELO_K= 32 — raised from 16 now that Elo *drives* sampling and so must track skill quickly),

$$r_\theta \leftarrow r_\theta + K(S - p_m), \quad r_m \leftarrow r_m - K(S - p_m) \quad (\text{anchored } r_m \text{ frozen}). \quad (36)$$

Because the learner starts at the **random** anchor’s rating and is updated against whatever it plays from iteration 0, r_θ self-calibrates and climbs the ladder. The update is zero-sum and the fixed anchors pin the absolute scale, so r_θ is an interpretable, drift-free skill curve (logged + plotted alongside win rate).

Pool management. After a short anchors-only warmup (SNAPSHOT_WARMUP iters, so the pool is not seeded with a useless random-init net) the learner is snapshotted into the pool every SELFPLAY_REFRESH iters (the new snapshot inherits r_θ); auto-snapshots are FIFO-pruned to LEAGUE_MAX_SNAPSHOTS. Externally supplied checkpoints (LEAGUE_INIT_CKPTS) are *pinned* (never pruned), letting past weights seed the pool. Snapshots live on CPU; only the sampled opponent is moved to the GPU for its rollout, so pool size costs RAM, not VRAM. A bounded pool of *historical* snapshots (vs. a single latest self) is what avoids the cycling/forgetting of naive self-play; the Elo/PFSP weighting then focuses compute on the most *informative* opponents. (Note: a discrete-actor learner cannot load a continuous- ϕ checkpoint — the ϕ head shape differs — so pre-discretization weights are skipped when seeding a discrete league.)

Native variant (win-gated mix). The native trainer’s league is the simpler win-gated form: opponents mixed from {**random**, **starter**, pool} with fixed weights (**mix_random**, **mix_starter**, **mix_pool**), the pool a bounded set (**pool_capacity**) of historical snapshots, and promotion *win-gated* — a new snapshot is admitted only when the current policy beats the pool at win rate

$\geq \text{promote_winrate}$. The notebook’s Elo/PFSP scheme generalizes this: rating-weighted sampling instead of a fixed mix, and a soft Elo ladder instead of a hard promotion gate.

10 Self-play league, curriculum, and reward shaping

This session extends the league of Section 9 into a full **scripted-anchor difficulty ladder** with an **Elo-gated curriculum**, a grounding **recalibration** that removes self-play Elo inflation, an **anti-clone / anchor-boosted matchmaking** rule, and a **production-weighted, asymmetric capture reward**. Where Section 9 described two anchors (**random**, **starter**) and “the league *is* the curriculum,” the ladder below supplies six anchors and a gated unlock; the PFSP machinery of Section 9 is the substrate, the additions here are the anchors, the gate, the recal, and the reward channel.

10.1 Scripted anchor bots (the difficulty ladder)

The pool is pinned by six hand-coded opponents, all **vectorized in the GPU env** (Section 11) so they cost no host round-trip. Each carries a fixed Elo that pins the rating scale (Section 9); the ratings are an *approximate, slightly conservative* ladder (see the round-robin note below). Throughout, “half-garrison” means half the source planet’s ship count, fired only when that planet holds ≥ 20 ships, and “lead-aimed” means the heading aims at the orbital *intercept* (**LEAD_TARGET**, Section 1) rather than the target’s current position.

Bot	Elo	Behavior
random	0	Random heading; launches half its garrison (≥ 20 ships) from any planet.
starter	600	Fires half-garrison (≥ 20) at the nearest static (non-rotating) non-owned planet, with a straight atan2 aim at its current position.
intermediate	750	In-flight-aware capture from the nearest <i>capable</i> owned source (cascading to the 2nd/3rd-nearest if closer ones cannot beat the target), lead-aimed at the intercept. Per target the ships needed are $\lfloor \text{garrison} \rfloor + \lfloor \text{enemy_inbound} \rfloor - \lfloor \text{friendly_inbound} \rfloor + \text{CAPTURE_MARGIN}$, where the inbound counts are the in-flight fleet ships attributed to that planet by the <i>same</i> fleet→target geometry as the threat encoder (Section 2). This (a) stops piling onto a target a friendly fleet already covers (needed $\leq 0 \Rightarrow$ skip) and (b) sends enough to <i>counter</i> an enemy fleet inbound to the same target (auto-recapture). Plus local rebalancing (send $\lfloor \text{gap}/2 \rfloor$ to the poorest owned planet within REBALANCE_RADIUS = 30 when surplus $> \text{REBALANCE_PCT} = 0.4$ of garrison) and comet-escape (an owned comet planet evacuates <i>all</i> its ships to the nearest owned non-comet planet before it expires).
greedy	900	Every owned planet independently captures the nearest beatable non-owned planet within CAPTURE_RADIUS = 25, sending $\lfloor \text{target_ships} \rfloor + \text{CAPTURE_MARGIN}$ (= 2), lead-aimed. <i>Note:</i> empirically the strongest scripted bot — its every-planet aggression + just-enough efficiency wins the board-grab race.
medium	1000	“starter++” : nearest non-owned planet <i>including</i> rotating ones, lead-aimed at the intercept, half-garrison.

(The **random**, **starter**, **intermediate**, **greedy**, **medium** rows are the five behavior families; **starter** differs from **medium** only by the static-vs-rotating target choice and the lead aim, and the listed

Elos pin the scale.) The shared constant is `CAPTURE_MARGIN = 2` (the over-send that beats combat round-off). **Empirical strength order** from a complementary round-robin in the official Kaggle engine is

`greedy > medium > intermediate > starter > random,`

so the anchor Elos above are an *approximate, slightly conservative* ladder (greedy is rated 900 yet is the practical ceiling).

10.2 Curriculum: Elo-gated anchor regimes

The anchors split into two regimes by skill, so a *weak* learner is not handed bots that would exploit and destabilize it before it can learn:

Base regime (always-active anchors): `random, starter, intermediate`. These are on from iteration 0.

Advanced regime (gated anchors): `greedy, medium`. Withheld early because they are too strong for a weak learner and would exploit / destabilize it.

Unlock mechanism (anti-inflation guard). When the learner’s online Elo first reaches the trigger `ADVANCE_TRIGGER_ELO = 800`, the trainer runs a *full recalibration with all anchors on* (including the gated ones) on **fresh random seeds**. If that honest recal grades the learner at $\geq \text{ADVANCE_CONFIRM_ELO} = 700$, the advanced anchors (`greedy, medium`) are *permanently* unlocked; otherwise the learner stays in the base regime — the 800 was self-play inflated (Section 10.3) and the gate refuses to be fooled. **Normally** (before any unlock) the periodic recalibration grounds the learner’s Elo against only the *active* anchors (`random, starter, intermediate`); the 800-trigger recal is the one moment all anchors are turned on at once.

10.3 Elo grounding, recalibration, and matchmaking

Logistic Elo back-solve (grounding against the anchors). Online self-play Elo (Section 9) inflates: the learner farms its own under-rated frozen snapshots, so r_θ drifts above true skill. The fix re-grounds r_θ against the *fixed* scripted anchors. For a measured score s (the **greedy** win $+\frac{1}{2}$ draw fraction, clamped to $[0.02, 0.98]$) against an anchor of rating R , the implied rating is the logistic inverse of the Elo expectation,

$$\hat{r}(R, s) = R + 400 \log_{10} \frac{s}{1-s}, \quad (37)$$

and the recalibrated r_θ is $\hat{r}$ *averaged over the active anchors*. This is run every `ELO_RECAL_EVERY` iterations on **fresh worlds**, which removes the online self-play inflation that otherwise accrues; the clamp $[0.02, 0.98]$ keeps $\log_{10}(s/(1-s))$ finite at a blow-out or a shut-out.

PFSP matchmaking weight. Each pool opponent’s sampling weight blends an Elo-based PFSP term with a behavioral-footprint term:

$$w_m = \text{MATCH_ELO_W} \cdot \underbrace{f(p_m)}_{\text{Elo PFSP}} + \text{MATCH_FP_W} \cdot \underbrace{\text{JS}(\hat{\pi}_m, \hat{\pi}_\theta)}_{\text{footprint dist.}} + \text{PFSP_FLOOR}, \quad (38)$$

with `MATCH_ELO_W = 0.7` and `MATCH_FP_W = 0.3`, where $f(p_m)$ is the Elo-based PFSP curve of Section 9 (the $p(1-p)/(1-p)^P$ weight on the logistic expectation p_m) and $\text{JS}(\hat{\pi}_m, \hat{\pi}_\theta)$ is the Jensen–Shannon distance between opponent m ’s and the learner’s **behavioral footprints**. The

footprint term is **anti-clone**: it down-weights near-identical snapshots (so the learner does not over-play copies of itself) and, since scripted anchors are maximally distinct from the learner, keeps them in the mix.

Scripted-anchor matchmaking boost. On top of Eq. (38), each scripted anchor’s weight is multiplied by `SCRIPT_MATCH_BOOST` = 2.0 (i.e. +100% — sampled twice as often) *until* the learner has mastered that bot, after which the boost drops to 1×. The boost-drop test uses a per-bot EMA win-rate $m[\mathbf{wr}]$ (EMA factor `WR_EMA_BETA` = 0.1): the boost drops once $m[\mathbf{wr}] \geq \text{SCRIPT_BOOST_DROP_WR}$ = 0.90 after at least `SCRIPT\_BOOST\_MIN\_GAMES` = 5 games against it. The purpose is to **concentrate rollouts on the bots the learner has not yet mastered**.

Knob	Symbol / use	Value
<code>ELO_RECAL_EVERY</code>	recalibration period (fresh worlds)	—
score clamp	$s \in [\cdot]$ for Eq. (37)	[0.02, 0.98]
<code>ADVANCE_TRIGGER_ELO</code>	online Elo that triggers the all-anchor recal	800
<code>ADVANCE_CONFIRM_ELO</code>	recal Elo needed to unlock greedy/medium	700
<code>MATCH_ELO_W</code> / <code>MATCH_FP_W</code>	Elo-PFSP vs. footprint blend, Eq. (38)	0.7 / 0.3
<code>SCRIPT_MATCH_BOOST</code>	scripted-anchor weight multiplier (until mastered)	2.0
<code>SCRIPT_BOOST_DROP_WR</code>	EMA win-rate that drops the boost to 1×	0.90
<code>SCRIPT_BOOST_MIN_GAMES</code>	games before the boost can drop	5
<code>WR_EMA_BETA</code>	EMA factor for the per-bot win-rate $m[\mathbf{wr}]$	0.1
<code>CAPTURE_MARGIN</code>	scripted over-send (all capture bots)	2

10.4 Reward shaping: production-weighted, asymmetric capture

The capture channel C of Section 5 is sharpened so the agent is pulled toward *valuable* targets and is rewarded for *net expansion*. Two changes:

Production-weighted capture. Capturing a planet of production Π pays `CAPTURE_REWARD` · (1 + `CAPTURE_PROD_SCALE` · Π), so higher-production planets are worth more. With `CAPTURE_REWARD` = 30 (tunable base) and `CAPTURE_PROD_SCALE` = 0.2, a prod-1 planet pays 1.2× base and a prod-5 planet 2.0× base.

Asymmetric loss. Losing a planet removes only `CAPTURE_LOSS_FRAC` = 0.9 of its (production-weighted) value, so a capture-then-recapture cycle is net +10% — an **anti-passivity** incentive to keep expanding rather than trade evenly.

The full per-step capture reward, masked to the **active (alive)** ego planets, is the following (writing w_c for `CAPTURE_REWARD`, ρ_Π for `CAPTURE_PROD_SCALE`, λ_ℓ for `CAPTURE_LOSS_FRAC`, and $n^\pm, \Pi^\pm$ for the count and summed-production gained (+) and lost (−) this step):

$$C_t = w_c \left[(n^+ + \rho_\Pi \Pi^+) - \lambda_\ell (n^- + \rho_\Pi \Pi^-) \right], \quad w_c = 30, \rho_\Pi = 0.2, \lambda_\ell = 0.9. \quad (39)$$

This generalizes the flat ± 30 form $C_t = w_c(\#gained - \#lost)$ of Section 5: setting $\rho_\Pi = 0$ and $\lambda_\ell = 1$ recovers it exactly.

Knob	Meaning	Value
<code>CAPTURE_REWARD</code>	base capture pay (tunable)	30
<code>CAPTURE_PROD_SCALE</code>	per-unit-production bonus (prod-1 → 1.2×, prod-5 → 2.0×)	0.2
<code>CAPTURE_LOSS_FRAC</code>	fraction of value removed on a loss (net +10%/cycle)	0.9

11 Batched GPU simulator

The naive rollout is *latency-bound*: each of the $T \approx 500$ ticks is a strict chain `encode (CPU) → forward (GPU) → decode+step (CPU)` with a host $\leftrightarrow$ device transfer on either side of the forward, so the GPU sits idle $\sim 70\%$ of the time waiting on the CPU simulation. `GpuEnv (native/rl/gpu_env.{hpp,cpp})`, enabled by `RolloutConfig::gpu_env` instead runs the *entire* game as batched LibTorch tensor ops on the GPU, so the whole loop — encode, sample, decode, step, scripted opponent — stays on-device. The deep-policy forward and the swept fleet/planet collision then dominate, making the loop **GPU-compute-bound** (observed 100% SM utilization), not CPU- or transfer-bound. It uses *pure* tensor ops (no custom CUDA kernel / toolkit).

Layout (Structure-of-Arrays, leading batch B). Planets live in $E_c = \text{planet_cap}$ *fixed* slots (the last 4 reserved for the at-most-one live comet group); fleets in a fixed pool of `fleet_cap` slots with an *alive* mask. Because the per-planet actor is permutation-equivariant (Section 3), **no entity_order sort is needed** — row order is irrelevant, which removes the one operation that does not vectorize cleanly. So here $E = E_c$ (the obs/action dim), independent of the serving-time top-40 cap. Integer quantities (ships, production, owner, step) are kept in `float32` (exact $< 2^{24}$).

Vectorized dynamics. The per-tick update mirrors the reference engine (`core/sim.hpp`) branch-free: orbital motion and production are elementwise; launches scatter into free fleet slots; the swept ray/disk collision is the $B \times \text{fleet_cap} \times E_c$ pairwise test (the heavy on-device compute). *Combat needs no sort*: with exactly two players, arriving ships scatter-add per owner into (B, E_c) , then top/second/survivor are elementwise (the reference’s stable-sort/first-seen order only matters for > 2 owners). Comets are a baked per-env schedule (spawn step, ships, precomputed path positions) overlaid each tick. The threat encoder (Section 2) breaks equal-ship ties by a per-fleet *launch-order* stamp to match the reference’s `stable_sort` over the fleet list.

Parity. `GpuEnv` is a third engine and must implement the same rules as the C++ `ow::step` that is also the Kaggle submission engine. `apps/parity_gpu` drives both from identical states/actions and compares per-env aggregates: the deterministic core is **bit-exact** (64 envs $\times$ many ticks), the observation encoder is parity-exact (max feature diff $\sim 10^{-6}$), the **starter** opponent is bit-exact, and full episodes with comets stay $\geq 90\%$ exact, the residual being benign `float32-vs-float64` boundary flips in marginal collisions (a training-time fidelity choice, not a logic error).

On-device rollout, host-offloaded trajectory. `GroupedRollout::collect_gpu` keeps the loop on-device but copies each step’s (obs, action, log-prob) to *pinned* host buffers with non-blocking transfers, so VRAM holds only the model, the current step’s working set, and the collision intermediate — bounded in T and B , i.e. **not memory-bound**. The reward channels (Section 5, bounded variant), terminal test, and group advantage are computed on-device; valid transitions then flatten into the same host `TrajectoryBatch` the CPU path produces, so the GRPO update (Section 7) is unchanged and streams minibatches back to the GPU. Within a group all envs share one (world, opponent) so the group baseline stays valid; the opponent is uniform across the batch per iteration.

Deep-net numerical stability. Summing the squashed-Gaussian log-prob over $E \cdot 3K$ components makes a deep, from-scratch policy prone to blow-ups: an extreme μ with a saturated sample (latent z far from μ) over a tiny σ^2 produces a huge $\log N$, hence an exploding step and

$\exp(\log p - \text{old}) \rightarrow \infty$. The trainer guards this with a tight log-std range $[\varsigma_{\min}, \varsigma_{\max}] = [-2, 1]$, a clamp on μ inside the density (the saturated region is unchanged), a clamp on the log-ratio before exp, a $[-10, 10]$ clamp on the whitened advantage (a near-degenerate group otherwise explodes it), and skipping any optimizer step whose gradient norm is non-finite.

11.1 Simulator fidelity (sim-to-real gap)

The vectorized GPU simulator (`GpuEnv`) was verified **bit-exact** against the official Kaggle engine (`kaggle_environments orbit_wars`) for steps 0–48 across many games of **starter** versus **starter**. The method replays *identical* moves through both engines and compares, each tick, planet ships/owner/position and the per-owner fleet counts. Every core mechanic matches: fleet launch (the surface offset of Section 1), production timing, planet orbit, the swept-pair continuous collision, combat resolution (top/second, ties, owner-flip), sun occlusion, and the fleet speed law

$$\text{speed} = \min\left(\text{shipSpeed}, 1 + (\text{shipSpeed} - 1)\left(\frac{\ln(\text{ships})}{\ln 1000}\right)^{1.5}\right), \quad (40)$$

which is exactly the fleet speed $v(\nu)$ of Section 2 (with $\text{shipSpeed} = v_{\max} = 6$). **The only divergence is comets:** the official engine spawns comets at steps 50/150/250/350/450 (random ship counts and paths), which `GpuEnv` omits — so comets are the *sole* sim-to-real gap. Empirically (official-engine eval) a current checkpoint scores ~ 0.91 vs. **random** but only ~ 0.04 vs. **starter**, with comets costing ~ 6 points; the dominant weakness is therefore **policy quality, not the simulator**.

12 Symbols $\leftrightarrow$ config.hpp

Symbol	config.hpp field	Default
d	ModelConfig::hidden	128
E	ModelConfig::max_entities	40
K	ModelConfig::fleets_per_planet (continuous/native only; leading target uses none)	5
τ_{act}	ModelConfig::act_threshold (fleet-commit)	0.05
use_glu	ModelConfig::use_glu (trunk A/B), n_{res}	true, 2
USE_ATTENTION	cross-planet self-attention block (notebook A/B ablation)	true
n_v	VALUE_RES_BLOCKS (critic depth; residual, same block as trunk)	2
$[\zeta_{\text{min}}, \zeta_{\text{max}}]$	ModelConfig log-std clip	$[-2, 1]$
$\zeta_{\text{max}}^{\text{end}}$	logstd_max_end (phase-1 forced-decay target)	$= \zeta_{\text{max}}^{\text{end}}$ (of
$\zeta_{\text{max}}^{\text{post}}$	logstd_max_post (phase-2 head cap)	$= \zeta_{\text{max}}^{\text{end}}$
N_{σ}	sigma_decay_iters (phase-1 length, iters)	0 (off)
std_state_dependent	ModelConfig (ζ head vs vector)	true
init_mu_scale	ModelConfig (calm-init mean-head weight scale)	0.02
init_phi_bias	ModelConfig (calm-init ϕ bias at step 0)	-4.0
action_mask_policy	ModelConfig (hard-mask actor to legal slots)	false
target_actor	ModelConfig (false $\Rightarrow$ continuous (d^x, d^y, ϕ) actor)	false
G, N	GrpoConfig::group_size, num_groups	8, 64
ϵ	GrpoConfig::clip	0.2
β	GrpoConfig::kl_beta	0.04
$c_{\mathcal{H}}$	GrpoConfig::ent_coef	0.01
ε	GrpoConfig::adv_eps	10^{-4}
w_d, w_o	GrpoConfig::dense_weight, outcome_weight	1, 1
γ	GrpoConfig::gamma	1.0
w_{Π}, P_{max}	prod_reward_weight, prod_reward_cap	2.5, 100
δ_{Π}	prod_reward_decay (per-step prod decay)	0.997
w_v, V_{max}	valid_launch_reward, valid_reward_cap	0.02, 30
ρ	illegal_launch_penalty (uncapped I)	0.005
ρ_m	miss_launch_penalty (uncapped M , aiming stick)	0.01
win/loss	win_bonus, loss_penalty (legacy default; run uses 1000)	300, 100
w_e	enemy_growth_weight (off in the run)	0.5
x_0, β	enemy_growth_warmup, enemy_growth_ema	50, 0.1
<i>set-ups/1.md event reward (defaults off; the run sets them):</i>		
δ_o	win_decay, loss_decay (run: 0.995)	1.0
L_0	decay_start_step (win/loss flat until this step; run: 100)	0
w_c	capture_reward (run: 30)	0
w_d, N_d	dispatch_reward, dispatch_count (run: 1, 50)	0, 50
w_m, base	prod_milestone_reward, prod_milestone_base (run: 20, 100)	0, 100
σ_R	ppo_reward_scale (=reward-scale; run: 100)	1.0
<i>PPO + GAE (PpoConfig; algo="ppo"):</i>		
λ	ppo.gae_lambda	0.95
γ_{ppo}	ppo.gamma (GAE discount)	0.99
c_v	ppo.vf_coef (value-loss weight)	0.5
selfplay_prob	RolloutConfig (stage-4 self-play fraction)	0.5
<i>Allocation actor + reachability + Elo league (notebook setup1_target_ppo.ipynb):</i>		
POLICY_MODE	"ppo" (per-source Dirichlet) vs. "mcts" (search priors; stub)	ppo
κ	ALLOC_KAPPA (Dirichlet concentration scale, $\alpha = \text{softplus}(\ell)\kappa + \epsilon$)	—
n_{min}	MIN_LAUNCH_SHIPS (below this, an off-diagonal entry is held not launched)	1
LEAD_TARGET	aim at orbital intercept (Section 1) vs. static target	true
REACH_MASK, $r_{\odot}$	sun-reachability mask; SUN_RADIUS disk	true, 10
$r_{\theta}(0), r_{\text{rand}}, r_{\text{start}}$	ELO_INIT; anchored ELO_RANDOM/ELO_STARTER	0; 0, 600
K	ELO_K (raised: Elo now drives sampling)	32
PFSP	PFSP_MODE, PFSP_FLOOR 27	even, 0.05
pool	LEAGUE_MAX_SNAPSHOTS, SELFPLAY_REFRESH	12, 100
no stages	TOTAL_ITERS, SNAPSHOT_WARMUP (league = curriculum)	1400, 100
stage	RolloutConfig::stage	2

A Appendix: PPO + GAE (the current default algorithm)

The `set-ups/1.md` run trains with **actor-critic PPO + GAE** (`TrainConfig::algo="ppo"`, which sets `ModelConfig::use_value_head`); GRPO of Section 7 is the alternative. PPO adds a value head V_ϕ (the critic: a masked-mean pool of the post-trunk tokens over live planets, concatenated with the global embedding g' , then an input projection $d+d_g \rightarrow d$, **VALUE.RES_BLOCKS pre-LayerNorm residual MLP blocks** — the *same* ResNet block as the trunk, $z \mapsto z + W_b \text{ReLU}(W_a \text{LN}(z))$, so every hidden MLP layer in the network is residual with the skip running all the way through — and a $d \rightarrow 1$ readout) and uses Generalized Advantage Estimation so the *per-step* reward $r_t = C_t + D_t + P_t$ of Section 5 (with the outcome O scattered onto the last step) reaches the policy gradient temporally — the credit assignment that GRPO discards by collapsing each trajectory to a single episode return. With TD residual $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ and the alive/done masks of the rollout,

$$\hat{A}_t^{\text{GAE}} = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l}, \quad \mathcal{L}^V = \frac{1}{2} (V_\phi(s_t) - \hat{R}_t)^2, \quad \hat{R}_t = \hat{A}_t^{\text{GAE}} + V_\phi(s_t), \quad (41)$$

with discount $\gamma = \text{ppo.gamma} = 0.99$ (GAE applies it, so the reward must *not* pre-discount) and trace decay $\lambda = \text{gae.lambda} = 0.95$; an env alive at the time cap bootstraps with $V_\phi(s_T)$, while a true death contributes no bootstrap. The total loss reuses the clipped surrogate (24), KL anchor (25), and entropy bonus of the GRPO objective with $\hat{A}_i$ replaced by $\hat{A}_i^{\text{GAE}}$, plus the value loss at weight $c_V = \text{vf.coef} = 0.5$:

$$\mathcal{L} = \mathcal{L}^{\text{clip}} + \beta \mathcal{L}^{\text{KL}} + c_V \mathcal{L}^V - c_{\mathcal{H}} \overline{\mathcal{H}}.$$

PPO reuses `GrpoConfig`'s `clip`, `ent_coef`, `update_epochs`, `minibatches`, and `kl_beta`. Conversely, GRPO (Section 7) removes V_ϕ (and thus λ , c_V , and value warmup), replacing $\hat{A}_i^{\text{GAE}}$ with the group-relative $\hat{A}_i$ of Eq. (22).

B Appendix: search-based policy improvement — the AlphaZero/MuZero family

PPO improves the policy with a *first-order* gradient on sampled returns. The AlphaZero/MuZero family instead improves it by *planning*: at each state a tree search produces a *locally improved* policy π' and a sharpened value, and the network is trained to imitate them. Discretizing the action (Section 4) was the prerequisite, so this appendix maps the family onto Orbit Wars: what transfers directly, the three obstacles our game poses, and which post-MuZero paper resolves each. This is the planned successor to PPO, not yet implemented.

B.1 What maps directly: AlphaZero, not MuZero

MuZero (Schrittwieser et al., 2020) learns a *model* — a representation h , a dynamics function g , and a reward head — precisely because Atari/real domains give no usable simulator. **We have the opposite situation:** `GpuEnv` (Section 11) is a perfect, deterministic, *batched and cheaply cloneable* simulator (state is flat SoA tensors; a node is a tensor slice copy). So the learned model is unnecessary — **AlphaZero** (Silver et al., 2018), which searches with the *real* model, is the template. The components line up as:

MuZero component	Orbit Wars analogue	Status
representation $h_\theta(o) \rightarrow s$	encoder X, g (Section 2) + trunk (Section 3)	exists
dynamics $g_\theta(s, a) \rightarrow s'$	the real <code>GpuEnv</code> step (perfect)	exists; not learned
reward head $r_\theta(s, a)$	the real event reward (Section 5)	exists; not learned
prediction policy $p_\theta(s)$	factored actor $\prod_{s:\text{owned}} \text{Dirichlet}(\alpha_s)$ (per-source allocation, Section 4)	exists
prediction value $v_\theta(s)$	value head V_ϕ (the PPO critic, Appendix A)	exists

Our network is therefore *already* an AlphaZero “prediction function”: a policy prior + a leaf value. Three things still stand between us and a working search.

B.2 Obstacle 1: a combinatorial (factored) action space

A turn commits one allocation simplex $A[s, \cdot] \in \Delta^{E-1}$ per *owned* planet, so even after discretizing each E -way row into a finite grid of allocations the joint action is the product $\prod_{s:\text{owned}} (\text{grid over } \Delta^{E-1})$ — with $E=48$ destinations per source and several owned sources this is astronomically large. AlphaZero’s enumerate-all-children PUCT is hopeless. The fix is the post-MuZero line for large/structured actions:

- **Sampled MuZero** (Hubert et al., 2021, “Learning and Planning in Complex Action Spaces”) samples m actions from a proposal β (take $\beta = \pi_\theta$, our factored prior — sampling the joint is just sampling each owned source’s allocation row) and searches over *only* those m . To stay unbiased it replaces the PUCT prior of a sampled action by $\hat{\beta}(a)/\beta(a) \cdot \pi_\theta(a)$ (with $\hat{\beta}$ the empirical sample frequency; for $\beta = \pi_\theta$ this is $\hat{\beta}$), so the sampled improvement equals the full-action improvement *in expectation*.
- **Gumbel MuZero** (Danihelka et al., 2022) samples *without* replacement via the Gumbel-Top- k trick, which is strictly more informative than Sampled MuZero’s with-replacement draws and — crucially — gives a *guaranteed* policy improvement at *any* simulation budget (Section B.3). This is our recommended root algorithm.
- **Multiagent Gumbel MuZero** treats exactly our case — a joint action factored over “agents” (here, owned planets) — by drawing the Gumbel-Top- k *per factor* and evaluating the sampled *joint* actions, so cost scales with the number of *samples*, not the size of the product space.

The reachability mask (Section 4) helps here too: it shrinks each source row’s destination support *before* sampling, so the m sampled joint actions are all legal and on-target.

B.3 Obstacle 2: very few simulations — Gumbel policy improvement

Because the net is deep and each “simulation” is a real (batched) env step plus a forward, we can afford only a handful of sims per state. Gumbel MuZero is built for exactly this. With prior logits $\text{logits}(a)$ and per-action search value $q(a)$:

Root sampling. Draw $g \sim \text{Gumbel}(0)$ and take the top- m actions $\mathcal{A}_{\text{top}m} = \text{argtop}_m(g(a) + \text{logits}(a))$ — equivalent to sampling m distinct actions from π_θ .

Budget allocation. Sequential Halving splits n sims over $\log_2 m$ phases: visit the current candidates equally, then drop the worse half by $g + \text{logits} + \sigma(\hat{q})$, repeat.

Action chosen / guarantee. $A^* = \arg \max_{a \in \mathcal{A}_{\text{top}m}} (g(a) + \text{logits}(a) + \sigma(q(a)))$, with the monotone scale

$$\sigma(\hat{q}(a)) = (c_{\text{visit}} + \max_b N(b)) c_{\text{scale}} \hat{q}(a), \quad c_{\text{visit}}=50, \quad c_{\text{scale}}=1. \quad (42)$$

This *provably improves* on the prior at any n : $q(A^*) \geq q(\arg \max_a (g(a) + \text{logits}(a)))$, hence $\mathbb{E}[q(A^*)] \geq \sum_a \pi_\theta(a)q(a)$ — unlike “act greedily on the top- n ,” which can *worsen* the policy.
Improved-policy target (completed Q). Define $\text{completed}Q(a) = q(a)$ if a was visited, else v_θ (the value approximation, with $v_\pi = \sum_a \pi_\theta(a)q(a)$); then the training target is

$$\pi' = \text{softmax}(\text{logits} + \sigma(\text{completed}Q)). \quad (43)$$

This labels *all* actions (not just A^*) and assigns unvisited actions zero advantage.

Non-root selection. Inside the tree, descend deterministically by $\arg \max_a (\pi'(a) - \frac{N(a)}{1 + \sum_b N(b)})$ (Grill et al., 2020, MCTS as regularized policy optimization) — visit the action whose visit share most lags π' .

Here each $q(a)$ is obtained by stepping the real env to the child and bootstrapping with V_ϕ (plus the one-step event reward of Section 5), i.e. a depth-1 (or shallow n -step) search; the value head we already train under PPO is the leaf evaluator.

B.4 Obstacle 3: simultaneous moves and a hidden opponent

AlphaZero assumes a *sequential, perfect-information* game; Orbit Wars is *simultaneous* (both sides act each tick) and from ego’s view the opponent’s move is hidden, so the transition $s \rightarrow s'$ under ego action a is *stochastic*. Two clean resolutions, both leaning on machinery we already have:

- **Opponent-as-chance (recommended).** The Stage-4 opponent is a *frozen league policy* μ we fully control (Section 9). Treat its move as a *chance event*: a search node is (ego decision) $\rightarrow$ (sample K_o opponent actions $a^o \sim \mu(\cdot | s)$) $\rightarrow$ (deterministic env step), and back up the *expectation* over a^o . This is precisely the *afterstate / chance-node* construction of **Stochastic MuZero** (Antonoglou et al., 2022) — except we need *no* learned chance model, because μ is a known network we can query. Averaging over a PFSP *mixture* of league opponents additionally makes π' a best response to the *population*, not to one snapshot (anti-cycling, as in Section 9).
- **Simultaneous-move search.** Alternatively, solve the per-node 2-player *matrix game* over the (sampled) joint actions — decoupled UCB / regret matching at each node (cf. ReBeL, Brown et al., 2020, for the imperfect-information generalization). Stronger game-theoretically (targets a Nash rather than a best response) but materially more complex; the opponent-as-chance route is the pragmatic first cut.

B.5 Targets, loss, and the training-loop change

Search converts each visited state into supervised targets, replacing PPO’s clipped surrogate:

$$\mathcal{L}_{\text{AZ}}(\theta) = \underbrace{\text{KL}(\pi' \| \pi_\theta)}_{\text{policy distill, Eq. (43)}} + c_V (V_\phi(s) - z)^2 + c_{\text{reg}} \|\theta\|^2, \quad (44)$$

where π' is the Gumbel improved policy (factorized back to the per-planet heads via the per-factor marginals of the sampled joint actions) and the value target z is the search root value (an n -step return bootstrapped by the backed-up tree value), or the game outcome O for terminal-anchored training. Two carry-overs from the family:

- **Reanalyze** (MuZero / EfficientZero, Ye et al., 2021): because the simulator is perfect and cheap, stored trajectories can be *re-searched* with the *latest* network to refresh (π', z) , turning

the on-policy rollout into reusable, near-on-policy data — a large sample-efficiency win and the main thing EfficientZero contributes that survives having a perfect model (its self-supervised consistency loss and value-prefix target are *model-learning* fixes we do not need).

- **The factored actor and both masks are unchanged;** only the *target* the heads chase changes (search π' instead of the PPO advantage). The value head, reachability/validity masks, GPU rollout, and Elo league all transfer as-is — the league in fact *supplies* the opponent distribution that Obstacle 3 plans against.

B.6 Minimal first cut for Orbit Wars

A concrete, low-risk instantiation: **Gumbel AlphaZero with sampled factored actions and opponent-as-chance**, depth-1 (value-bootstrapped) to start. Per training state: (1) sample m joint actions from the masked factored prior via per-factor Gumbel-Top- k ; (2) for each, sample K_o league opponent moves, step `GpuEnv`, and score $q = \bar{r} + \gamma V_\phi(s')$ averaged over a^o ; (3) pick A^* and form π' by Eq. (43); (4) train heads to π' and V_ϕ to the backed-up value. This reuses every existing piece, needs only a state snapshot/restore on `GpuEnv` and a small search loop, and degrades gracefully to current PPO when $m=1$ (no search). Deepening the tree (multi-ply sequential halving) and upgrading opponent-as-chance to a simultaneous-move solve are the follow-ons.